

ANDREAS SCHÄFER

THEORETISCHE INFORMATIK

TH LÜBECK

Danksagungen

Für das Finden von Tippfehlern danke ich Tom Böttger.

Inhaltsverzeichnis

1	<i>Reguläre Sprachen und endliche Automaten</i>	7
2	<i>Kontextfreie Sprachen und Kellerautomaten</i>	29
3	<i>Berechenbarkeit und Komplexität</i>	45

1

Reguläre Sprachen und endliche Automaten

FUSSGÄNGER-AMPELN ZEIGEN ENTWEDER ROT ODER GRÜN für die Fußgänger und die entsprechend andere Farbe für die Autofahrer. Sie haben zwei Zustände und wechseln dazwischen. Komplexere Ampelsteuerungen können mit der gleichen Idee realisiert werden, man benötigt lediglich mehr Zustände. Das gleiche Konzept mit Zuständen und Übergängen dazwischen findet man bei vielen Systemen, vom Kaffeeautomaten über CPUs bis hin zum TCP-Protokoll. Diese sogenannten endlichen Automaten sind leicht zu verstehen, können für einfache Systeme gut graphisch dargestellt werden und sind auch noch sehr einfach in Hard- oder Software zu implementieren. Das ist der Grund für ihre hohe Verbreitung.

Diese endlichen Automaten arbeiten auch im Hintergrund bei der Analyse von Nutzereingaben, von einfachen Webformularen bis hin zu Compilern, die Programmcode in Maschinencode übersetzen.



Abbildung 1.1: Einfaches Modell einer Fußgängerampel

1.1 Alphabet, Wort, Sprache

Alphabet, Wort und Sprache sind bekannte Begriffe. Für die Informatik sind sie besonders wichtig. Programme müssen Eingaben von Nutzern oder anderer Programmen verarbeiten. Der Programmcode selbst ist eine ein "Wort" einer Programmiersprache. Deshalb lohnt es sich, die intuitiv bekannten Begriffe einmal genauer zu betrachten.

Definition 1: Alphabet

Ein **Alphabet** Σ ist eine endliche und nicht-leere Menge. Die Elemente des Alphabets heißen Buchstaben.

Definition 2: Wort

Ein **Wort** $w = a_1 \dots a_n$ ist eine Sequenz von Buchstaben eines Alphabets.

Definition 3: Sprache

Eine **Sprache** L (Language) ist eine endliche oder unendliche Menge von Wörtern.

Alphabet, Wort, Sprache (Alphabet, Word, Language)



<https://youtu.be/y0xop6RaHvs>
(<https://youtu.be/A5BZQkNZ7zs>)

Aus Programmiersprachen kennt man typischerweise die leere Zeichenkette als Sonderfall und das Verketteten (Hintereinanderschreiben oder Konkatenieren) als Operation auf Zeichenketten. Wenn man mehrfach das gleiche Wort hintereinander schreibt gelangt man zur Potenz.

Definition 4: Operationen auf Wörtern

- ε bezeichnet das **leere Wort**
- $w_1 \cdot w_2$ oder $w_1 w_2$ bezeichnet das Hintereinanderschreiben von w_1 und w_2 (**Konkatenation**)
- w^n bezeichnet das Wort $\underbrace{w \dots w}_{n\text{-mal}}$ wobei $w^0 = \varepsilon$.
- $|w|$ ist die **Länge** des Wortes w
- Σ^* ist die Menge aller Wörter über dem Alphabet Σ (inkl. ε).

Die Operationen auf Wörtern kann man zu Operationen auf Sprachen erweitern.

Definition 5: Konkatenation und Potenz von Sprachen

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$ (Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$
 Alternativ: $L^{n+1} = L^n \circ L$
 Menge aller Wörter, die man erhalten kann, wenn man n Wörter aus L zusammensetzt.

Sie kennen von Dateimasken einen Ausdruck wie $*.doc$. Dahinter verbirgt sich der Sternoperator, der erlaubt beliebig viele Wörter einer Sprache hintereinander zu hängen. Hier sind sogar Null Wörter erlaubt. In dem Beispiel $*.doc$ steht der Stern ganz analog für eine beliebige Anzahl von Buchstaben, die hintereinander geschrieben sind.

Definition 6: Stern

Sei L eine Sprache. Dann ist der **Sternabschluss** definiert durch

$$L^* = \bigcup_{i=0}^{\infty} L^i.$$

Es ist die Menge aller Wörter, die man erhalten kann, wenn man beliebig viele (auch Null) Wörter aus L zusammensetzt. Das leere Wort ε ist immer in L^* enthalten.

Für den Fall, dass man mehr als Null Wörter hintereinander erwartet, gibt es den Plusoperator.

Definition 7: Plus

Sei L eine Sprache. Dann ist der **Plusabschluss** definiert durch

$$L^+ = \bigcup_{i=1}^{\infty} L^i.$$

Es ist die Menge aller Wörter, die man erhalten kann, wenn man ein oder mehr Wörter aus L zusammensetzt. Das leere Wort ε ist in L^+ nur enthalten, wenn es schon in L enthalten ist.

1.2 Deterministische Endliche Automaten

Ein großes Anwendungsgebiet der Automatentheorie liegt im Bereich der formalen Sprachen. Mit Hilfe von endlichen Automaten werden Nutzereingaben analysiert und geprüft. Dazu liest der Automat das Wort Zeichen für Zeichen ein und “merkt” sich im aktuellen Zustand sein Wissen über die bis jetzt gelesene Eingabe.

Die graphische Darstellung wie in Abbildung 1.2 ist intuitiv gut verständlich. Um mit Automaten arbeiten und sie in Algorithmen verarbeiten zu können, nutzt man eine Definition eines Automaten als Struktur. Solche Strukturen sind die mathematischen Vorfahren Strukturen in C und der Klassen, die sie aus Java kennen.

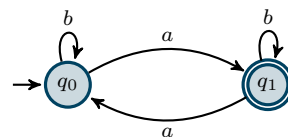


Abbildung 1.2: DEA A mit $L(A) = \{w \in \{a, b\}^* \mid \#_a(w) \text{ ist ungerade}\}$

Definition 8: DEA / DFA

Ein **deterministischer endlicher Automat** (engl. *deterministic finite automaton*), kurz **DEA** (engl. *DFA*), ist eine Struktur $(Q, \Sigma, \delta, q_0, F)$ mit folgenden Eigenschaften:

- Q ist eine endliche Menge von **Zuständen** (states)
- Σ ist eine endliche Menge, das **Eingabealphabet** (input alphabet)
- $\delta: Q \times \Sigma \rightarrow Q$ ist die **Überföhrungsfunktion** (transition function)
- $q_0 \in Q$ ist der **Anfangszustand** (start / initial state)
- $F \subseteq Q$ ist die Menge der **Endzustände oder akzeptierenden Zustände** (final / accepting states)

Endliche Automaten lesen Zeichen für Zeichen der Eingabe, nehmen dazu jeweils eine Kante (Transition) und passen so den Zustand an. Nachdem der Automat alle Zeichen gelesen hat, schaut man, ob der letzte Zustand ein akzeptierender Zustand ist oder nicht. Wenn wir uns für ein Eingabewort die Zustände merken, erhält man einen Lauf.

Definition 9: Lauf/Run eine DEA

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Ein **Lauf** (Run) von A für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- q_0 der Startzustand von A ist und

Endliche Automaten (Finite Automata)



https://youtu.be/FGccyPXY_x4
(<https://youtu.be/9hsXXGabi8>)

Endliche Automaten – Formale Definition (Finite Automata – Formal Definition)



<https://youtu.be/bJtM3Ze1yvc>
(<https://youtu.be/5FMku9912p4>)

- $\delta(q_{i-1}, a_i) = q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Falls ein solcher Lauf existiert, schreiben wir $q_0 \xrightarrow{w} q_n$. Bei DEAs gibt es für jedes Wort **genau einen** Lauf. Ein Lauf heißt **akzeptierend**, falls er in einem akzeptierenden Zustand endet, d.h. $q_n \in F$.

Definition 10: Akzeptierte Sprache eines DEA

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q_f \text{ und } q_f \in F\}$$

Definition 11: Reguläre Sprache

Eine Sprache heißt **regulär** genau dann wenn es einen deterministischen endlichen Automaten gibt, der diese Sprache akzeptiert.

DEAs konstruieren

Wenn man für ein gegebenes Problem einen DEA konstruieren möchte, sollte man zuerst überlegen, was sich der DEA über die Eingabe für das konkrete Problem merken sollte.

- Falls der Automat prüfen soll, ob das Wort das Teilwort *aba* enthält, müsste er sich merken, wie viel der Automat vom Wort schon gesehen hat.
- Falls der Automat prüfen soll, ob die Wörter keine *c* enthalten, muss er sich im Zustand merken, ob er ein *c* gesehen hat. Falls ja, muss er in einem Zustand sein aus dem kein akzeptierender Zustand erreichbar ist.
- Falls der Automat prüfen soll, ob die Anzahl der *a* größer als 4 ist, muss er "mitzählen", ob er null, ein, zwei, drei, vier oder mehr als vier *a* gesehen hat. Ob es 5 oder 15 sind, spielt für das Problem und damit für den Automaten dann keine Rolle.
- Falls der Automat prüfen soll, ob die Anzahl der *a* durch 4 teilbar ist, muss er sich den aktuellen Rest bei Division der Anzahl der *a* durch 4 merken. Eine einfache Unterscheidung zwischen den Fällen, "ist durch 4 teilbar" und "ist nicht durch 4 teilbar" genügt nicht, weil "ist nicht durch 4 teilbar" könnten 3 *a* oder 1 *a* sein. Im ersten Fall wäre die Anzahl beim nächsten *a* durch 4 teilbar und im zweiten Fall nicht.

In vielen Fällen kann man einfach mit einem Startzustand starten, überlegen was der Automat für das leere Wort im Bezug auf das Problem weiß und dann für jeden Eingabebuchstaben den Automaten Schritt für Schritt erweitern.

DEAs können allgemeine Eigenschaften von Wörtern prüfen. Sie können sogar in begrenztem Maß zählen. ‘

Wie DEAs arbeiten (How DFAs work)



<https://youtu.be/3iS5wB9toFc>
(<https://youtu.be/gqmxCPKF-VE>)

Erster Buchstaben = Letzter Buchstabe (First Letter = Last Letter)



<https://youtu.be/kXX-cyup7Wg>
(<https://youtu.be/kbadZjUug6k>)

DEAs zählen (DFAs count)



<https://youtu.be/DFE0kR5ybbk>
(https://youtu.be/MLiFRW_sANE)

1.3 Nichtdeterministische Endliche Automaten

Bei den deterministischen endlichen Automaten (DEA) ist das Verhalten für jede Eingabe genau festgelegt. Für jeden Zustand und jedes Eingabezeichen steht der Nachfolgezustand fest. Man kann den Automaten mehr Freiheit geben und eine Wahlmöglichkeit geben. So kommt man zu den nichtdeterministischen endlichen Automaten. Diese Automaten können in einem Zustand für eine Eingabe mehr als einen Nachfolger haben oder auch gar keinen.

Für ein Eingabewort hat ein NEA dann ggf. verschiedene mögliche Läufe. Davon können einige erfolgreich in einen Endzustand führen und andere nicht. Uns werden nur die Läufe interessieren, die in einem Endzustand führen. Falls ein NEA eine Auswahlmöglichkeit hat und die eine Möglichkeit ihn irgendwann in einen Endzustand bringt und die andere nicht, so entscheidet er sich immer für die, die in den Endzustand führt.

Diese Automaten erlauben Eigenschaften einfacher und übersichtlicher zu beschreiben. Sie eignen sich deshalb besonders für Spezifikationen und für spätere Konstruktionen.

An der formalen Definition müssen wir nicht viel ändern, wir erlauben lediglich eine Menge von Startzuständen und eine Menge von Nachfolgezuständen für jeden Zustand und Eingabebuchstaben.

Definition 12: NEA

Ein **nichtdeterministischer endlicher Automat** (engl. *nondeterministic finite automaton*), kurz **NEA** (engl. *NFA*), ist eine Struktur $A = (Q, \Sigma, \delta, Q_0, F)$ mit folgenden Eigenschaften:

- Q ist eine endliche Menge von **Zuständen**
- Σ ist eine endliche Menge, das **Eingabealphabet**
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ ist die **Überföhrungsfunktion**, die für jeden Zustand und Buchstaben eine Menge von Nachfolgezuständen liefert.
- $Q_0 \subseteq Q$ ist eine nicht-leere Menge von **Anfangszuständen**
- $F \subseteq Q$ ist die Menge der **Endzustände**.

Die Definition von Lauf und Akzeptanz sind fast bei deterministischen und nichtdeterministischen endlichen Automaten fast identisch. Wesentlicher Unterschied ist, dass es im deterministischen Fall genau einen Lauf gibt.

Definition 13: Lauf/Run eine NEA

Sei $A = (Q, \Sigma, \delta, Q_0, F)$ ein NEA. Ein **Lauf** (Run) von A für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- $q_0 \in Q_0$ ein Startzustand von A ist und
- $\delta(q_{i-1}, a_i) \ni q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Falls ein solcher Lauf existiert, schreiben wir $q_0 \xrightarrow{w} q_n$. Bei NEAs gibt es

NEAs: Automaten, die raten (Introducing Guessing: Nondeterministic Finite Automata)



<https://youtu.be/Bv-QHc6KI0U>
(https://youtu.be/MLiFRW_sANE)

NEAs: Formale Definition (NEAs: Formal Definition)



<https://youtu.be/9LQFDv22akY>
(<https://youtu.be/U--WfHVooyE>)

für jedes Wort **keinen, einen oder mehr** Läufe. Ein Lauf heißt **akzeptierend**, falls er in einem akzeptierenden Zustand endet, d.h. $q_n \in F$.

Durch die Möglichkeit auszuwählen, welchen Weg NEAs nehmen möchten, sind sie eine Verallgemeinerung der DEAs. Tatsächlich können NEAs aber nicht mehr als DEAs. Man kann für jeden NEA einen äquivalenten DEA konstruieren. Die grundlegende Idee dabei ist, dass sich der DEA im Zustand merkt, in welchen Zuständen der NEA gerade sein könnte. Davon gibt es nur endlich viele Möglichkeiten.

Theorem 14: Rabin und Scott – Potenzmengenkonstruktion

Sei $A_N = (Q_N, \Sigma, \delta_N, Q_{0N}, F_N)$ ein nichtdeterministischer endlicher Automat. Konstruiere einen deterministischen endlichen Automaten $A_D = (\mathcal{P}(Q), \Sigma, \delta_D, Q_{0N}, F_D)$ mit

- $\mathcal{P}(Q_N) = \{S \mid S \subseteq Q_N\}$, d.h. die Zustände von A_D sind mit Zustandsmengen von A_N beschriftet
- $\delta_D(\{q_1, \dots, q_k\}, a) = \bigcup_{i=1}^k \delta_N(q_i, a)$, d.h. der deterministische Nachfolgezustand von $\{q_1, \dots, q_k\}$ mit Buchstabe a ist die Menge aller Zustände, die der NEA A_N von einem Zustand in $\{q_1, \dots, q_k\}$ mit a erreichen kann.
- $F_D = \{S \subseteq Q \mid S \cap F_N \neq \emptyset\}$, d.h. ein Zustand ist akzeptierend, falls er einen akzeptierenden Zustand des NEAs enthält.

Dann akzeptieren DEA A_D und NEA A_N die gleiche Sprache.

Das Problem bei der Konstruktion ist, dass die Zustandszahl exponentiell wächst. In vielen Fällen sind aber bei weitem nicht alle diese Zustände vom Startzustand aus erreichbar. Solche Zustände müssten gar nicht konstruiert werden. Das erreicht man dadurch, dass man schrittweise vom Startzustand aus die Konstruktion wie in Algorithmus 1.3 durchführt.

1.4 Abschlusseigenschaften

Es gibt eine Reihe von interessanten Konstruktionen, die uns helfen endliche Automaten für unsere Probleme zu finden. Wenn man bereits Automaten für Teilprobleme hat, helfen diese Konstruktionen Automaten zur Lösung komplexerer Probleme zusammenzusetzen.

Durch diese Konstruktionen zeigt man gleichzeitig, dass die von endlichen Automaten akzeptierbaren Sprachen unter diesen Operationen abgeschlossen sind. Für zwei natürliche Zahlen ist deren Summe auch immer eine natürliche Zahl. Man sagt in diesem Fall, dass die natürlichen Zahlen unter der Operation Summe abgeschlossen sind. Bei den Operationen auf Sprachen betrachten wir neben den einfachen Operationen aus der Mengenlehre auch Operationen, die die Wörter ändern. Eine schöne Eigenschaft der Sprachen von endlichen Automaten ist, dass sie unter sehr vielen Operationen abgeschlossen sind.

Vom NEA zum DEA Potenzmengenkonstruktion (From NEA to DFA - Subset Construction)



<https://youtu.be/1ZhMqCQ9UUg>
(<https://youtu.be/zombtiFPXac>)

Abschlusseigenschaften (Closure Properties)



<https://youtu.be/pwbKUDd0oZE>
(<https://youtu.be/WCC9WrHS99s>)

Abschluss unter Vereinigung (Closure under Union)



https://youtu.be/7Qg_jreqi-s
(<https://youtu.be/0iICVHMQRd0>)

Abschluss unter Vereinigung - Formale Definition (Closure under Union – Formal Definition)



<https://youtu.be/XLwHrF1FIgw>
(<https://youtu.be/He9V7GNe2A8>)

Algorithmus: Potenzmengenkonstruktion

Input: NEA $A_N = (Q_N, \Sigma, \delta_N, Q_{0N}, F_N)$
Output: DEA $A_D = (Q_D, \Sigma, \delta_D, Q_{0N}, F_D)$

```

1:  $Q_D \leftarrow \{Q_{0N}\}$            ▷ Beginne mit Startzustand des DEA: ist mit Startzustandsmenge des NEA beschriftet
2:  $Waiting \leftarrow \{Q_0\}$        ▷ Liste der unbearbeiteten Zustände enthält Startzustand
3: while  $Waiting \neq \emptyset$  do           ▷ Solange es unbearbeitete Zustände gibt
4:    $Q \leftarrow \text{NEXTSTATE}(Waiting)$    ▷ Wähle unbearbeiteten Zustand  $Q$  des DEAs aus Warteliste aus.
5:   for all  $x \in \Sigma$  do           ▷ Gehe alle Buchstaben  $x$  des Alphabets durch
6:      $Q' \leftarrow \emptyset$ 
7:     for all  $q \in Q$  do           ▷ Für jeden Zustand  $q$  des NEAs, der im Zustand  $Q$  des DEAs vorkommt.
8:        $Q' \leftarrow Q' \cup \delta_N(q, x)$    ▷ Sammle alle  $x$ -Nachfolger von  $q$  im NEA in Menge  $Q'$  auf.
9:     end for
10:    if  $Q' \notin Q_D$  then           ▷ Falls  $Q'$  im DEA noch nicht vorkommt
11:       $Q_D \leftarrow Q_D \cup \{Q'\}$            ▷ füge ihn hinzu
12:       $Waiting \leftarrow Waiting \cup \{Q'\}$    ▷ und setze ihn auf die Warteliste
13:    end if
14:     $\delta_D \leftarrow \delta_D \cup \{Q \xrightarrow{x} Q'\}$    ▷ Füge  $x$ -Kante von  $Q$  nach  $Q'$  in DEA hinzu.
15:  end for
16:   $F_D \leftarrow \emptyset$ 
17:  for all  $Q \in Q_D$  do           ▷ Gehe alle Zustände  $Q$  des DEAs durch
18:    if  $Q \cap F_N \neq \emptyset$  then   ▷ Falls  $Q$  einen Endzustand des NEAs enthält
19:       $F_D \leftarrow F_D \cup \{Q\}$            ▷ Mache  $Q$  im DEA zum Endzustand.
20:    end if
21:  end for
22: end while

```

Abbildung 1.3: Algorithmus zur Potenzmengenkonstruktion. Es werden nur die erreichbaren Zustände des DEAs konstruiert.

Abschluss gegen Vereinigung

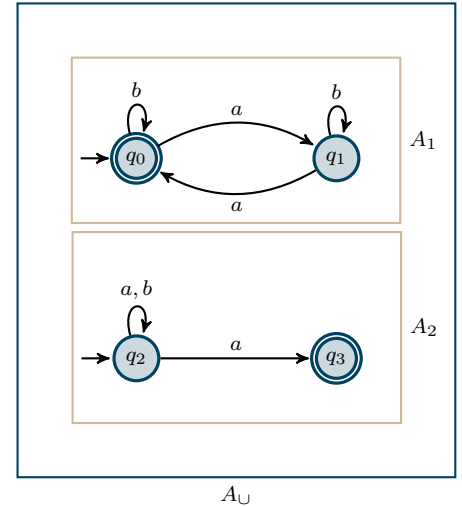
Alternativen entsprechen der Vereinigung von Sprachen. Aus zwei Automaten kann ganz einfach ein Automat konstruiert werden, indem die Automaten vereinigt werden. Der so entstehende Automat ist immer nichtdeterministisch und entscheidet am Anfang, ob er sich wie der erste oder der zweite Automat verhalten möchte.

Theorem 15: Abschluss gegen Vereinigung

Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ nichtdeterministische endliche Automaten. Sei $A_{\cup} = (Q_{\cup}, \Sigma, \delta_{\cup}, Q_{0\cup}, F_{\cup})$ definiert durch

- $Q_{\cup} = Q_1 \cup Q_2$ (Anmerkung: disjunkte Vereinigung)
- $F_{\cup} = F_1 \cup F_2$
- $Q_{0\cup} = Q_{01} \cup Q_{02}$
- $\delta_{\cup} = \delta_1 \cup \delta_2$

Dann gilt: $L(A_{\cup}) = L(A_1) \cup L(A_2)$.



Abschluss gegen Komplement

Im Komplement einer Sprache sind genau die Wörter, die nicht in der Ausgangssprache sind. Wenn wir einen DEA haben, können wir einfach Nicht-Endzustände und Endzustände tauschen. Bei einem NEA funktioniert dieser Ansatz nicht in jedem Fall, da der NEA für ein Wort sowohl einen akzeptierenden als auch einen nicht akzeptierenden Lauf haben kann. Falls wir von einem NEA ausgehen, müssen wir deshalb mit Hilfe der Potenzmengenkonstruktion zuerst einen äquivalenten DEA konstruieren.

Theorem 16: Abschluss gegen Komplement

Sei A ein NEA, der die Sprache $L(A)$ akzeptiert. Dann gibt es einen DEA $A_D = (Q_D, \Sigma, \delta_D, q_{0D}, F_D)$ mit $L(A) = L(A_D)$. Der Automat $\bar{A} = (Q_D, \Sigma, \delta_D, q_{0D}, Q \setminus F_D)$ akzeptiert dann das Komplement, d.h. $(L(\bar{A}) = \Sigma^* \setminus L(A_D))$.

Abschluss unter Komplement (Closure under Complement)

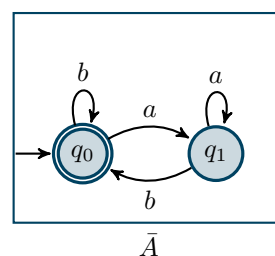
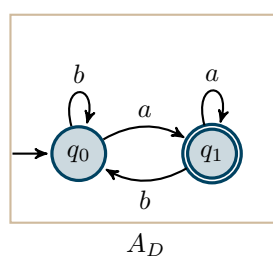
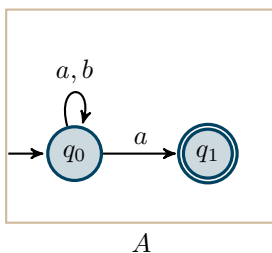


<https://youtu.be/5JPKSYk98s0>
(<https://youtu.be/0wHtNEZ6PBI>)

Abschluss unter Komplement - Formale Definition (Closure under Complement – Formal Definition)



https://youtu.be/8IbKc_Qlj0c
(<https://youtu.be/K7ap1YCxW3c>)



Abschluss gegen Schnitt

Aus der Mengenlehre kennen wir die Beziehung

$$L(A_1) \cap L(A_2) = \overline{\overline{L(A_1)} \cup \overline{L(A_2)}}.$$

Da wir schon gezeigt haben, wie Vereinigung und Komplement konstruiert werden können, haben wir dadurch eine Konstruktion für den Schnitt. Allerdings erfordert diese Konstruktion dreimal die Potenzmengenkonstruktion für das Komplement. Das ist sehr aufwändig. Sehr viel einfacher ist es, wenn wir einen Kreuzprodukt-Automaten konstruieren, der sich im Zustand merkt, in welchem Zustand jeweils die beiden Ausgangsautomaten gerade sind. Jeder Zustand ist dort mit einem Paar beschriftet, dessen erste Komponente der aktuelle Zustand des ersten Automaten und dessen zweite Komponente der aktuelle Zustand des zweiten Automaten ist. Falls ein Automat steckenbleibt, bleibt auch der Kreuzprodukt-Automat stecken.

Theorem 17: Abschluss gegen Schnitt

Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ nicht-deterministische endliche Automaten. Sei

$$A_\times = (Q_\times, \Sigma, \delta_\times, Q_{0\times}, F_\times)$$

definiert durch

- $Q_\times = Q_1 \times Q_2$
- $\delta_\times((q_1, q_2), a) = \delta_1(q_1, a) \times \delta_2(q_2, a)$
- $Q_{0\times} = Q_{01} \times Q_{02}$
- $F_\times = F_1 \times F_2$.

Dann gilt: $L(A_\times) = L(A_1) \cap L(A_2)$.

Für dieses Beispiel beweisen wir exemplarisch die Korrektheit der Konstruktion.

Beweis 18: Korrektheit der Produktkonstruktion

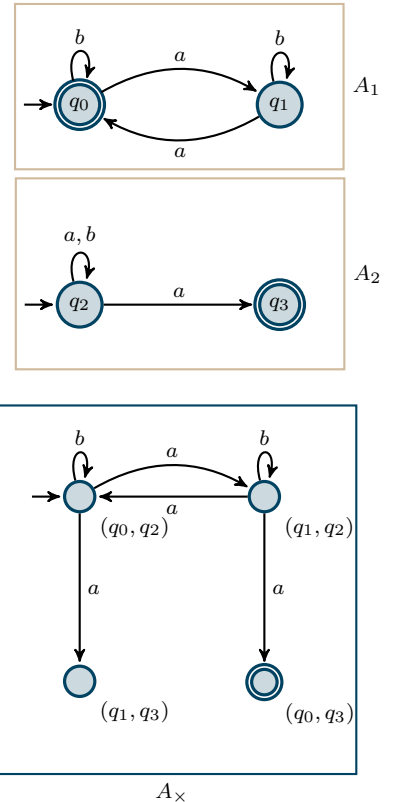
Wir wollen die Behauptung $L(A_\times) = L(A_1) \cap L(A_2)$ beweisen. Bei diesem Beweis wird die Gleichheit von zwei Mengen gezeigt. Das macht man praktisch dadurch, dass man zweimal eine Teilmengenbeziehung beweist.

$\subseteq$ -Richtung: Sei $w = a_1 \dots a_n \in L(A_\times)$. Dann gibt es nach Definition einen akzeptierenden Lauf

$$(q_{01}, q_{02})(q_{11}, q_{12})(q_{21}, q_{22}) \dots (q_{n1}, q_{n2})$$

in $A_\times$. Für diesen Lauf gilt

1. $(q_{01}, q_{02}) \in Q_\times = Q_{01} \times Q_{02}$ und damit $q_{01} \in Q_{01}$ und $q_{02} \in Q_{02}$
2. $\delta_\times((q_{i-1}1, q_{i-1}2), a) \ni (q_{i1}, q_{i2})$ und deshalb nach Definition von $\delta_\times$ auch $q_{i1} \in \delta_1(q_{i-1}1)$ und $q_{i2} \in \delta_2(q_{i-1}2)$ und



Abschluss unter Schnitt (Closure under Intersection)



<https://youtu.be/P4Iqr03-3X8>
(<https://youtu.be/hq6Y3gCyufU>)

3. $(q_{n1}, q_{n2}) \in F_{\times} = F_{01} \times F_{02}$ und damit $q_{n1} \in F_1$ und $q_{n2} \in F_2$.

Aus diesen Eigenschaften folgt, dass

$$q_{01}q_{11}q_{21} \dots q_{n1}$$

ein akzeptierender Lauf für w in A_1 ist und

$$q_{02}q_{12}q_{22} \dots q_{n2}$$

ein akzeptierender Lauf für w in A_2 ist. Also ist $w \in L(A_1) \cap L(A_2)$.

$\supseteq$ -Richtung: Sei $w \in L(A_1) \cap L(A_2)$. Dann gibt es akzeptierende Läufe

$$q_{01}q_{11}q_{21} \dots q_{n1}$$

für w in A_1 und

$$q_{02}q_{12}q_{22} \dots q_{n2}$$

für w in A_2 . Da es sich um akzeptierende Läufe der Automaten handelt, gilt

1. $q_{01} \in Q_{01}$ und $q_{02} \in Q_{02}$
2. $q_{i1} \in \delta_1(q_{(i-1)1})$ und $q_{i2} \in \delta_2(q_{(i-1)2})$ und
3. $q_{n1} \in F_1$ und $q_{n2} \in F_2$.

Nach Definition des kartesischen Produkts und von $A_{\times}$ folgt daraus

1. $(q_{01}, q_{02}) \in Q_{01} \times Q_{02} = Q_{\times}$
2. $(q_{i1}, q_{i2}) \in \delta_1(q_{(i-1)1}) \times \delta_2(q_{(i-1)2}) = \delta_{\times}((q_{(i-1)1}, q_{(i-1)2}), a)$
3. $(q_{n1}, q_{n2}) \in F_1 \times F_2 = F_{\times}$

Deshalb ist

$$(q_{01}, q_{02})(q_{11}, q_{12})(q_{21}, q_{22}) \dots (q_{n1}, q_{n2})$$

ein akzeptierender Lauf für w in $A_{\times}$ und damit $w \in L(A_{\times})$.

Abschluss unter Schnitt - formale Definition (*Closure under Intersection – Formal Definition*)



<https://youtu.be/vtPiB09JTRo>
(<https://youtu.be/DARTXcLPIW8>)

Beweis der Korrektheit der Produktkonstruktion (*Correctness of the Product Construction*)



<https://youtu.be/y9gw5KjDHDM>
(https://youtu.be/F39QI6eNL_8)

Gleichheit von Mengen beweisen

Um zu beweisen, dass zwei Mengen A und B gleich sind, zeigt man getrennt die Teilmengenbeziehungen $A \subseteq B$ und $B \subseteq A$. Für den Beweis von $A \subseteq B$ nimmt man ein beliebiges Element aus A an und zeigt, dass es auch in B liegt. Für den Beweis $B \subseteq A$ macht man das genau umgekehrt.

Beweisen, dass ein Automat ein Wort akzeptiert

Um zu zeigen, dass ein Automat ein Wort akzeptiert, muss man einen akzeptierenden Lauf konstruieren.

Abschluss unter Konkatenation

Die Konkatenation von zwei Sprachen enthält alle Wörter bei denen der erste Teil in der ersten Sprache und der zweite Teil in der zweiten Sprache liegt.

Um ausgehend von Automaten für zwei Sprachen einen Automaten für deren Konkatenation zu konstruieren, kann man beide Automaten in Reihe schalten. Hierbei kann der Wechsel vom ersten in den zweiten Automaten nichtdeterministisch ausgewählt werden. Anstatt in einen Endzustand des ersten Automaten zu wechseln kann der Automat sich entscheiden stattdessen in einen Startzustand des zweiten Automaten zu gehen.

Theorem 19: Abschluss unter Konkatenation

Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ endliche Automaten. Sei $A_\bullet = (Q_\bullet, \Sigma, \delta_\bullet, Q_{0\bullet}, F_\bullet)$ definiert durch

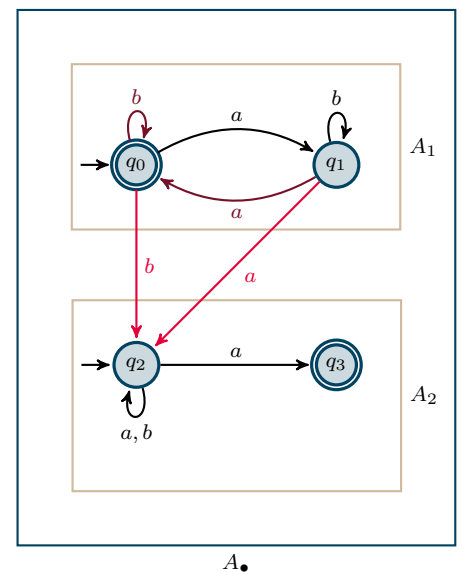
- $Q_\bullet = Q_1 \cup Q_2$ (Anmerkung: disjunkte Vereinigung)
- $F_\bullet = F_2$
- $Q_{0\bullet} = Q_{01}$ falls $Q_{01} \cap F_1 = \emptyset$ und $Q_{0\bullet} = Q_{01} \cup Q_{02}$ sonst.
- $\delta_\bullet(q, a) = \begin{cases} \delta_1(q, a) & \text{falls } q \in Q_1 \text{ und } \delta(q, a) \cap F_1 = \emptyset \\ \delta_1(q, a) \cup Q_{02} & \text{falls } q \in Q_1 \text{ und } \delta(q, a) \cap F_1 \neq \emptyset \\ \delta_2(q, a) & \text{falls } q \in Q_2 \end{cases}$

Dann gilt: $L(A_\bullet) = L(A_1) \cdot L(A_2)$.

Abschluss unter Konkatenation (Closure under Concatenation)



<https://youtu.be/Td03Ke8A-34>
(https://youtu.be/_hgtmsgFCKk)



Abschluss unter Stern

Im Sternabschluss einer Sprache befinden sich alle Wörter, die man erhält indem man beliebig viele Wörter der Ausgangssprache hintereinander schreibt. Im Automaten kann man diese Idee durch eine Rückkopplung realisieren. Anstatt in einen akzeptierenden Zustand zu wechseln, kann der Automat nichtdeterministisch auswählen in einen Startzustand zurückzugehen.

Theorem 20: Abschluss unter Stern

Sei $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ ein endlicher Automat. Sei $A_* = (Q_*, \Sigma, \delta_*, Q_{0*}, F_*)$ definiert durch

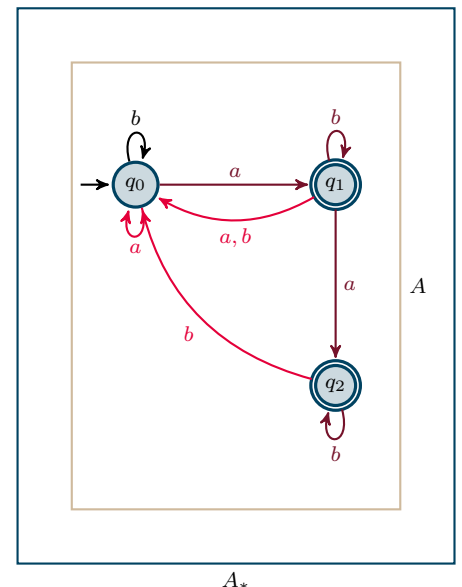
- $Q_* = Q_1 \cup \{q_\varepsilon\}$
- $F_* = F_1 \cup \{q_\varepsilon\}$
- $Q_{0*} = Q_{01} \cup \{q_\varepsilon\}$
- $\delta_*(q, a) = \begin{cases} \emptyset & \text{falls } q = q_\varepsilon \\ \delta(q, a) & \text{falls } q \neq q_\varepsilon \text{ und } \delta(q, a) \cap F_1 = \emptyset \\ \delta(q, a) \cup Q_{01} & \text{falls } q \neq q_\varepsilon \text{ und } \delta(q, a) \cap F_1 \neq \emptyset \end{cases}$

Dann gilt: $L(A_*) = L(A_1)^*$.

Abschluss unter Stern (Closure under Star)



<https://youtu.be/51PD0NaLjxs>
(<https://youtu.be/f-dTs5H5-uI>)



1.5 Reguläre Ausdrücke

Reguläre Ausdrücke sind Muster mit denen Sprachen beschrieben werden können. Alle kennen sie aus Dateidialogen in der Form *.doc. Programmiersprachen bieten häufig Funktionen zum Umgang mit solchen Ausdrücken an. Damit kann man Textdateien oder Nutzereingaben analysieren.

Reguläre Ausdrücke in Java

```
import java.util.Scanner;
import java.util.regex.Pattern;

//...
public static void MACExample () {
    Scanner scanner = new Scanner(System.in);
    // regular expression for MAC addresses
    String macRegex =
    ↪ "[0-9A-Fa-f]{2}[:-]{5}([0-9A-Fa-f]{2})";
    String input = scanner.nextLine();
    //Ex. b8:68:8f:27:6f:40
    if (Pattern.matches(macRegex, input)) {
        System.out.println("ok");
    }
}
```

Was sind reguläre Ausdrücke?

Wir wollen in diesem Kurs keine komplexen regulären Ausdrücke konstruieren. Es geht stattdessen darum zu verstehen, wie Java zum Beispiel prüfen kann, ob eine Eingabe einen regulären Ausdruck erfüllt oder welche Eigenschaften überhaupt prinzipiell durch reguläre Ausdrücke dargestellt werden können. Deshalb beschränken wir uns hier auf eine minimale Syntax, damit wir die regulären Ausdrücke gut analysieren können. Da unsere minimale Syntax in der Praxis zu sehr langen Ausdrücken führen würde, werden dort jede Menge Abkürzungen eingeführt, die aber nicht zu höherer Ausdrucksmächtigkeit führen.

Definition 21: Reguläre Ausdrücke

Wir definieren die regulären Ausdrücke E und die von ihnen beschriebene Sprache $L(E)$ induktiv;

- Induktionsanfang
 - $\emptyset$ ist ein regulärer Ausdruck und es gilt $L(\emptyset) = \emptyset$.
 - ε ist ein regulärer Ausdruck und es gilt $L(\varepsilon) = \{\varepsilon\}$.
 - Für alle Buchstaben des Alphabets a ist a ein regulärer Ausdruck und es gilt $L(a) = \{a\}$
- Induktionsschritt
 - Wenn E und F reguläre Ausdrücke sind, dann ist

Einfache Reguläre Ausdrücke in Java (Simple Regular Expressions in Java)



<https://youtu.be/-6GdqJ7aH8w>
(<https://youtu.be/0jdtAubZeTQ>)

Reguläre Ausdrücke in Java (Regular Expressions in Java)



<https://youtu.be/H-NcJ6GrJqg>
(<https://youtu.be/ufMT6gb2EdQ>)

Reguläre Ausdrücke (Regular Expressions)



<https://youtu.be/cYulnm8rTK4>
(<https://youtu.be/9IWbFL9u-fk>)

Reguläre Ausdrücke – Definition (Regular Expressions – Formal Definition)



<https://youtu.be/ciKhQ6CZDK0>
(<https://youtu.be/h9h-EQTDjVQ>)

- $(E + F)$ ein regulärer Ausdruck und es gilt $L(E + F) = L(E) \cup L(F)$,
- (EF) ein regulärer Ausdruck und es gilt $L(EF) = L(E) \circ L(F)$,
- E^* ein regulärer Ausdruck und es gilt $L(E^*) = L(E)^*$.

Um Klammern einzusparen vereinbaren wir, dass Konkatenation stärker bindet als Vereinigung.

Algebraische Regeln für Reguläre Ausdrücke

Reguläre Ausdrücke können algebraisch umgeformt werden, wie das auch bei Zahlen möglich ist. Dadurch können Ausdrücke vereinfacht werden. Einige Regeln sind genau wie bei Zahlen, einige Regeln sind sehr ähnlich zu schon bekannten Regeln.

Theorem 22: Kommutativität, Assoziativität und Distributivität

Seien E, F, H reguläre Ausdrücke. Dann gilt

- $E + F = F + E$,
- $E + (F + H) = (E + F) + H$,
- $E(FH) = (EF)H$,
- $E(F + H) = EF + EH$ und $(E + F)H = EH + FH$.

Theorem 23: Identitäten

Sei E ein reguläre Ausdruck. Dann gilt

- $\emptyset + E = E$,
- $\varepsilon E = E\varepsilon = E$,
- $\emptyset E = E\emptyset = \emptyset$.

Theorem 24: Regeln mit Stern

Sei E ein regulärer Ausdruck. Dann gilt

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,
- $\varepsilon^* = \varepsilon$,
- $E^+ \stackrel{def}{=} EE^*$,
- $E^+ + \varepsilon = E^*$.

Algebraische Regeln für Reguläre Ausdrücke (Algebraic Laws for Regular Expressions)



<https://youtu.be/DXl13wqJHzc>
(<https://youtu.be/AAud20tjtNk>)

Äquivalenz von Regulären Ausdrücken und Endlichen Automaten

Reguläre Ausdrücke sind gut geeignet zu beschreiben, welche Wörter erlaubt sind. Es fehlt noch eine gute Möglichkeit zu prüfen, ob ein

Wort zu einem regulären Ausdruck passt, d.h. in der vom regulären Ausdruck definierten Sprache liegt.

Java bietet dazu die Methode `Pattern.matches(regex, input)` an. Die Dokumentation zu dieser Methode spricht von einem NEA-basierten (engl. NFA-based) Verfahren. Es gibt also einen Zusammenhang mit endlichen Automaten. Tatsächlich sind die Sprachen, die von regulären Ausdrücken beschrieben werden können genau die Sprachen, die auch von endlichen Automaten akzeptiert werden können. Der Vorteil von regulären Ausdrücken ist, dass Sprachen einfach beschrieben werden können. Der Vorteil von endlichen Automaten ist, dass sie einfach zu implementieren sind und wir mit Ihrer Hilfe gut prüfen können, ob ein Wort in der Sprache liegt.

Es ist deutlich anschaulicher von einem regulären Ausdruck zu einem endlichen Automaten zu kommen als umgekehrt. Wir zeigen diese Richtung zuerst. Dies ist auch die Richtung, die wir in der Praxis brauchen, um zu prüfen, ob ein Wort zu einem Ausdruck passt.

Theorem 25: Kleene - Hinrichtung

Für jeden regulären Ausdruck E gibt es einen nichtdeterministischen endlichen Automaten A mit

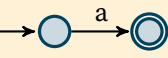
$$L(E) = L(A).$$

Der Beweis ist überraschend einfach. Wir können einfach der Struktur der Ausdrücke folgen und die schon bekannten Konstruktionen nutzen.

Beweis 26: Kleene – Hinrichtung

Für einen regulären Ausdruck E kann ein NEA A mit $L(E) = L(A)$ induktiv konstruiert werden:

- Basis:

- Automat für $\emptyset$: 
- Automat für ε : 
- Automat für a : 

- Induktionsvoraussetzung: Für reguläre Ausdrücke E_1 und E_2 , gibt es NEAs A_1 und A_2 mit $L(A_1) = L(E_1)$ und $L(A_2) = L(E_2)$.

- Induktionsschritt:

- $E_1 + E_2$: Konstruiere NEA für Vereinigung von $L(A_1)$ und $L(A_2)$
- $E_1 \cdot E_2$: Konstruiere NFA für Konkatination von $L(A_1)$ und $L(A_2)$
- E_1^* : Konstruiere NEA für $L(A_1)^*$

Die andere Richtung von einem Automaten zu einem regulären Ausdruck ist deutlich schwieriger. Es gibt eine Reihe von Verfahren,

Vom Regulären Ausdruck zum NEA (From Regular Expression to NFA)



<https://youtu.be/W3AZLHRumTE>
(<https://youtu.be/soIjd20hENY>)

Vom NEA zum regulären Ausdruck - Idee Zustandselimination (NFA To Regular Expression by State Elimination (Idea))



<https://youtu.be/vK0-eSnsfm8>
(<https://youtu.be/4tFXe74xMEM>)

Vom NEA zum regulären Ausdruck - Zustandselimination (Algorithmus) (NFA To Regular Expression by State Elimination (Algorithm))



<https://youtu.be/kIub1Fo2JtE>
(<https://youtu.be/rytI5C4Stj4>)

Vom NEA zum regulären Ausdruck - Lösen von Gleichungen (NFA to Regular Expression by Solving Equations)



<https://youtu.be/riIPwqlDceA>
(<https://youtu.be/rLfGAIHyxiM>)

wir stellen hier zwei Möglichkeiten vor. Da reguläre Ausdrücke keine Negation und keinen Schnitt enthalten, kann es schwierig sein, einen regulären Ausdruck zu bauen, der beschreibt, dass etwas nicht der Fall ist. In so einem Fall kann es einfacher sein, einen Automaten zu konstruieren und den regulären Ausdruck daraus abzuleiten. In der Praxis kommt das allerdings nicht so häufig vor.

Die andere Richtung ist trotzdem sehr relevant, weil nur beide Richtungen zusammen zeigen, dass reguläre Ausdrücke und endliche Automaten die gleiche Ausdrucksstärke besitzen. Diese Beziehung können wir später nutzen, um zu sehen, welche Eigenschaften mit regulären Ausdrücken *nicht* beschrieben werden können.

Theorem 27: Kleene - Rückrichtung

Für jeden nichtdeterministischen endlichen Automaten A gibt es einen regulären Ausdruck E mit

$$L(A) = L(E).$$

Der folgende Beweis funktioniert dadurch, dass für den Automaten ein Gleichungssystem aufgestellt und dann gelöst wird. Die rekursiven Gleichungen können wir dabei durch Lemma 29 lösen.

Beweis 28: Kleene - Rückrichtung

Sei $A = (Q, \Sigma, \delta, Q_0, F)$ ein NEA.

1. Für jeden Zustand $q_i \in Q$ führe Variable X_i und eine Gleichung:

(a) Seien $q_i \xrightarrow{a_1} q_{i_1}, \dots, q_i \xrightarrow{a_m} q_{i_m}$ die Transitionen, die in q_i starten.

(b) Falls $q_i \notin F$ (q_i ist nicht akzeptierend), führe die Gleichung ein

$$X_i = a_1 X_{i_1} + \dots + a_m X_{i_m}.$$

(c) Falls $q_i \in F$ (q_i ist akzeptierend), führe die Gleichung ein

$$X_i = a_1 X_{i_1} + \dots + a_m X_{i_m} + \varepsilon.$$

2. Das System wird durch Substitution und Ardens Lemma gelöst.

3. Der reguläre Ausdruck für $L(A)$ ist die Vereinigung (+) aller Lösungen für alle Startzustände.

Lemma 29: Ardens Lemma

Seien E, F, X reguläre Ausdrücke mit $\varepsilon \notin L(E)$. Dann hat die Gleichung $X = EX + F$ die Lösung $X = E^*F$.

Da reguläre Ausdrücke und endlichen Automaten die gleiche Klasse von Sprachen beschreiben, nennt man diese Klasse auch einfach reguläre Sprachen. Wir ergänzen deshalb Definition 11.

Definition 30: Reguläre Sprachen

Eine Sprache heißt regulär, wenn Sie durch einen regulären Ausdruck definiert werden kann. Dies ist genau dann der Fall, wenn sie durch einen NEA oder DEA erkannt werden kann.

1.6 Entscheidbare Eigenschaften regulärer Sprachen

Endliche Automaten und damit auch reguläre Sprachen sind sehr gut zu analysieren. Die einfachste Frage dabei ist, ob ein Automat ein Wort akzeptiert. Das ist leicht zu prüfen, da wir den Automaten direkt implementieren können.

Wir untersuchen eine Reihe von sogenannten Problemen und zeigen, dass sie für reguläre Sprachen entscheidbar sind.

- Wir beginnen bei dem einfachen Wortproblem, bei dem es nur darum geht, zu prüfen, ob ein Wort in der Sprache liegt.
- Beim Leerheitsproblem geht es darum zu prüfen, ob überhaupt ein Wort in der Sprache liegt. Das Verfahren verwendet Tiefensuche und ist auch die Grundlage für weitere Verfahren.
- Beim Enthaltenseinsproblem geht es darum zu prüfen, ob eine Sprache Teilmenge einer anderen Sprache ist. Diese Problem hat zum Beispiel Anwendungen in der automatischen Verifikation.
- Beim Äquivalenzproblem geht es darum zu prüfen, ob zwei Sprachen gleich sind. Mit dem vorgestellten Verfahren kann zum Beispiel geprüft werden, ob $(a+b)^*$ die gleiche Sprache beschreibt wie $(a^*b^*)^*$.

Definition 31: Wortproblem

Sei L eine Sprache und w ein Wort. Das Problem

$$w \in L$$

heißt Wortproblem.

Theorem 32: Wortproblem für reguläre Sprachen

Das Wortproblem für reguläre Sprachen ist entscheidbar.

Beweis 33: Wortproblem für reguläre Sprachen

Das Wortproblem für reguläre Sprachen kann wie folgt entschieden werden:

1. Umwandeln von A in DFA
2. Lauf von A für w simulieren
3. Falls Lauf in akzept. Zustand endet, gilt $w \in L(A)$.

Entscheidbare Eigenschaften regulärer Sprachen (Decidable Properties of Regular Languages)



<https://youtu.be/ghlqMe5CDnA>
(<https://youtu.be/0mUTLLkHiEw>)

Definition 34: Leerheitsproblem

Sei L eine Sprache. Das Problem

$$L = \emptyset$$

heißt Leerheitsproblem.

Theorem 35: Leerheitsproblem für reguläre Sprachen

Das Leerheitsproblem für reguläre Sprachen ist entscheidbar.

Beweis 36: Leerheitsproblem für reguläre Sprachen

Das Leerheitsproblem für reguläre Sprachen kann wie folgt entschieden werden:

1. Tiefensuche von Startzuständen zu akzeptierenden Zuständen.
2. Besuchte Zustände markieren dabei, um Zyklen zu erkennen.

Definition 37: Enthaltenseinsproblem

Seien L_1 und L_2 Sprachen. Das Problem

$$L_1 \subseteq L_2$$

heißt Enthaltenseinsproblem.

Theorem 38: Enthaltenseinsproblem für reguläre Sprachen

Das Enthaltenseinsproblem für reguläre Sprachen ist entscheidbar.

Beweis 39: Enthaltenseinsproblem für reguläre Sprachen

Seien A_1 und A_2 endliche Automaten. Da reguläre Sprachen unter Komplement und Schnitt abgeschlossen ist kann die Beziehung

$$L(A_1) \subseteq L(A_2) \iff L(A_1) \cap \overline{L(A_2)} = \emptyset$$

und die Entscheidbarkeit des Leerheitsproblems nach Theorem 35 genutzt werden und das Enthaltenseinsproblem wie folgt entschieden werden:

1. A_2 komplementieren
2. Ergebnis mit A_1 schneiden
3. Auf Leerheit testen

Definition 40: Äquivalenzproblem

Seien L_1 und L_2 Sprachen. Das Problem

$$L_1 = L_2$$

heißt Äquivalenzproblem.

Theorem 41: Äquivalenzproblem für reguläre Sprachen

Das Äquivalenzproblem für reguläre Sprachen ist entscheidbar.

Beweis 42: Äquivalenzproblem für reguläre Sprachen

Da das Enthaltenseinsproblem nach Theorem 38 entscheidbar ist, ist das Äquivalenzproblem wie folgt entscheidbar:

1. Prüfe $L(A_1) \subseteq L(A_2)$
2. Prüfe $L(A_2) \subseteq L(A_1)$

1.7 Grenzen regulärer Sprachen und das Pumping-Lemma

Endliche Automaten haben nur ihre Zustandsmenge, um sich Informationen über die Eingabe zu “merken”. Dieser Speicher ist endlich und wenn die Eingabe sehr lang ist, können endliche Automaten irgendwann die Eingaben nicht mehr präzise unterscheiden. Bei einer einfachen Sprache wie $\{a^n b^n \mid n \in \mathbb{N}\}$ muss ein Automat beim einlesen der a deren Anzahl mitzählen, um prüfen zu können, ob die Anzahl der später kommenden b gleich ist. Das können endliche Automaten jedoch nicht, da ihr Speicher irgendwann erschöpft ist.

Das Pumping-Lemma für reguläre Sprachen

Das Pumping-Lemma ist die Standardmethode, um zu zeigen, dass eine Sprache nicht regulär ist. Es handelt sich dabei um eine Eigenschaft, die alle regulären Sprachen besitzen. Wenn man also zeigen kann, dass die Eigenschaft nicht gilt, folgt daraus, dass die Sprache nicht regulär sein kann. Es gibt allerdings Sprachen, die nicht regulär sind, aber trotzdem die Eigenschaften des Lemmas erfüllen.

Lemma 43: Pumping-Lemma

Sei L regulär. Dann gibt es ein $p \in \mathbb{N}$, so dass jedes Wort $z \in L$ mit $|z| \geq p$ zerlegt werden kann in $z = uvw$ mit

1. $|v| \geq 1$
2. $|uv| \leq p$
3. Für alle $i \in \mathbb{N}$ (inkl. 0): $uv^i w \in L$

Für den Beweis des Pumping-Lemmas nutzt man die Idee, dass ein endlicher Automat beim Akzeptieren von sehr langen Wörtern eine Schleife durchlaufen muss. Diese Schleife kann dann aber auch mehrfach durchlaufen werden.

Beweis 44: Pumping-Lemma

Sei L regulär. Dann gibt es einen Automaten, der L akzeptiert. Der Automat hat eine endliche Anzahl $p \in \mathbb{N}$ an Zuständen. Sei $z = a_1 \dots a_n \in L$ mit $|z| \geq p$. Da das Wort z in der Sprache ist, muss

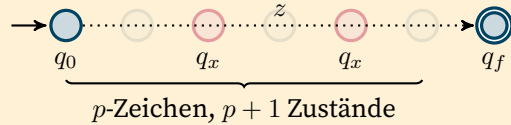
Grenzen regulärer Sprachen (Limits of Finite Automata)

<https://youtu.be/gMUtjesEdYY>
(<https://youtu.be/cFIASPeVQU>)

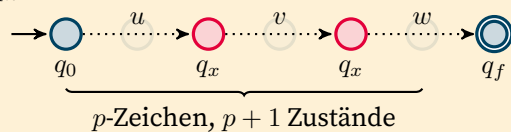
Das Pumping-Lemma (The Pumping Lemma)

<https://youtu.be/CquKIOoofj0>
(<https://youtu.be/cFIASPeVQU>)

der Automat das Wort akzeptieren mit einem akzeptierenden Lauf $q_0 \dots q_n$. Nach Einlesen der ersten p Zeichen von z enthält der Lauf bereits $p + 1$ Zustände. Unter diesen ersten $p + 1$ Zuständen muss mindestens ein Zustand q_x doppelt vorkommen, da der Automat nach Voraussetzung nur p Zustände besitzt.



Die Eingabe z besteht also aus den Teilwörtern $z = uvw$, wobei u das Teilwort ist, dass vom Startzustand zu q_x führt, v das Teilwort ist, dass vom q_x zu q_x führt und w das Teilwort ist, dass vom q_x zu q_n führt.



Da schon nach den ersten p Zeichen ein Zustand doppelt vorkommen muss erhalten wir die Eigenschaften:

1. $|v| \geq 1$
2. $|uv| \leq p$.

Der Automat besitzt also einen Zyklus von q_x zu q_x . Dieser Zyklus kann auch weggelassen oder mehrfach durchlaufen werden. In diesen Fällen gelangt der Automat aber ebenfalls in einen akzeptierenden Zustand. Also gilt für alle $i \geq 0$, dass $uv^i w$ in der Sprache L liegt.

Mit dem Pumping-Lemma kann man zeigen, dass eine Sprache nicht regulär ist. Das bedeutet, dass es weder einen endlichen Automaten gibt, der die Sprache akzeptiert, noch einen regulären Ausdruck, der die Sprache beschreibt. Wir demonstrieren das Vorgehen an der Sprache $\{a^n b^n \mid n \in \mathbb{N}\}$. Völlig analog funktioniert das Verfahren aber auch für Kammersprachen. Hier muss die Anzahl der öffnenden Klammern mit der Anzahl der schließenden Klammern verglichen werden. Deshalb gibt es keinen regulären Ausdruck, der korrekte Klammerungen zum Beispiel in arithmetischen Ausdrücken oder für Blockstrukturen in Programmiersprachen beschreibt.

Beweis 45: $\{a^n b^n\}$ ist nicht regulär.

Annahme: $L = \{a^n b^n\}$ ist regulär und p die Konstante nach Pumping-Lemma. Betrachte das Wort $z = a^p b^p$. Wegen $z \in L$ und $|z| \geq p$ hat es nach Pumping-Lemma eine Zerlegung $z = uvw$. Wegen $|uv| \leq p$ kann uv und damit v nur a s enthalten, d.h. v hat die Form $v = a^k$ für ein k und wegen $|v| \geq 1$ gilt muss $k \geq 1$ gelten. Wir pumpen ab und betrachten $uv^0 w = uw = a^{p-k} b^p \notin L$. Dies ist ein Widerspruch zum Pumping-Lemma und L ist deshalb nicht regulär.

$L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ ist nicht regulär ($L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ is not regular)



<https://youtu.be/1bcfP0Xw0Gg>
(<https://youtu.be/28ffi04Y9cY>)

$L = \{a^n \mid n \text{ ist prim}\}$ ist nicht regulär ($L = \{a^n \mid n \text{ ist prim}\}$ is not regular)



https://youtu.be/FPNzxUg_z4w
(<https://youtu.be/9ZLBesYo7HU>)

Erstellen von Pumping-Lemma-Beweisen

1. Wir nehmen an, dass die Sprache regulär ist und nennen die Pumping-Lemma-Konstante p . Wir dürfen keinen festen Wert für dieses p annehmen und arbeiten einfach mit der Variablen weiter.

Wir nehmen an, dass $L = \{w \in \{a, b\}^ \mid \#_a(w) > \#_b(w)\}$ regulär ist und p die Konstante nach Pumping-Lemma ist.*

2. Wir suchen ein Wort z aus der Sprache:

- (a) Das Wort muss mindestens p Zeichen enthalten, d.h. das p muss irgendwie in der Beschreibung des Wortes vorkommen. Der einfachste Fall ist, dass p in das Pattern eingesetzt wird.

Ungeeignet wäre $z = aaaabb$, weil wir nicht wissen, ob das Wort mindestens p Zeichen hat.

- (b) Das Wort muss möglichst speziell gewählt werden. Es sollten keine Variablen vorkommen, die nicht in einer Beziehung zu p stehen.

Ungeeignet wäre $z = w$ mit $\#_a(w) > \#_b(w)$ und $|w| > p$, weil wird jetzt keine Ahnung haben, wie das Wort aussieht.

Ungeeignet wäre $z = a^{p+1+k}b^p$, weil die Variable k uns das Argument schwer macht.

Besser ist $z = a^{p+1}b^p$, weil wir genauer wissen, wie das Wort aussieht.

- (c) Das Wort sollte möglichst an der "Grenze" der Sprache sein, damit eine kleine "Veränderung" schon aus der Sprache herausführt.

Ungeeignet wäre $z = a^{p+5}b^p$, weil wir nicht wissen, wie groß später das v sein wird und wir deshalb nicht wissen, ob abpumpen wirklich aus der Sprache führt.

Besser ist $z = a^{p+1}b^p$, weil wir mit nur einem a weniger schon aus der Sprache raus sind.

- (d) Das Wort sollte möglichst einfach sein, wir sollte möglichst genau wissen, wie die ersten p -Zeichen des Wortes lauten.

Ungeeignet wäre $z = aa^{\lceil \frac{p}{2} \rceil} b^{\lceil \frac{p}{2} \rceil}$, weil die ersten p Buchstaben unterschiedlich sind und wir jetzt nicht folgern können, welche Buchstaben in v vorkommen.

Besser ist $z = a^{p+1}b^p$, weil die ersten p Zeichen sicher alle a sind.

- (e) Das Ziel ist, durch Pumpen innerhalb der ersten p -Zeichen des Wortes aus der Sprache herauszukommen.

3. Wir wissen jetzt, dass das Wort eine Zerlegung $z = uvw$ hat. Die Zerlegung selbst dürfen wir nicht wählen, aber wir haben das Wort so geschickt gewählt, dass wir aus dem Bedingungen $|uv| \leq p$ und $|v| > 0$ ableiten können, wie v ungefähr aussehen muss.

Bei dem Wort $z = a^{p+1}b^p$ wissen wir, dass es eine Zerlegung hat $z = uvw$ und, dass dann v nur a s enthalten kann.

- Wir pumpen das v auf oder ab, um ein Wort zu erzeugen, was nicht in der Sprache liegt. Auf diese Weise erzeugen wir einen Widerspruch zur Aussage des Pumping-Lemmas. Wir dürfen einmal abpumpen oder beliebig oft aufpumpen. Das dürfen wir wählen.

Aufpumpen von v erhöht die Anzahl der a im Wort noch weiter und führt nicht aus der Sprache heraus. Abpumpen verringert die Anzahl um mindestens ein a und führt aus der Sprache raus. Wir pumpen deshalb ab.

- Aufgrund des Widerspruchs zum Pumping-Lemma schließen wir, dass die Sprache nicht regulär ist.

Beweis 46: $L = \{a^n \mid n \text{ ist Quadratzahl}\}$ ist nicht regulär.

Wir nehmen an, L ist regulär. Dann gibt es nach Pumping-Lemma ein p mit den dort beschriebenen Eigenschaften.

Betrachte das Wort $a^{(p^2)}$. Das Wort ist länger als p , also zerlegbar $a^{(p^2)} = uvw$ mit $|uv| \leq p$. Dann muss nach Pumping-Lemma (3) aber $uv^2w = a^{(p^2)+|v|}$ auch in der Sprache sein. Dazu müsste $p^2 + |v|$ (mit $1 \leq |v| \leq p$) eine Quadratzahl sein. Die nächstgrößere Quadratzahl über p ist aber $(p+1)^2 = p^2 + 2p + 1$. Widerspruch. Also war die Annahme falsch, dass L regulär ist.

1.8 Der Minimierungsalgorithmus

Deterministische Endliche Automaten kann man minimieren. Die Idee dabei ist, dass zwei Zustände eines DEA genau dann nicht äquivalent sind, wenn man von dem einen Zustand mit einem Wort in einen Endzustand gelangt und von dem anderen Zustand mit dem gleichen Wort nicht.

Anders formuliert sind Zustände äquivalent, wenn man mit jedem Wort von dem einen Zustand genau dann in einen Endzustand kommt, wenn man auch von dem anderen Zustand aus in einen Endzustand kommt.

Diese Idee liefert den Algorithmus zur Minimierung. Wir finden nach und nach alle nicht äquivalenten Zustandspaare. Danach verschmelzen wir die äquivalenten Paare zu einem Zustand.

Algorithmus: Der Minimierungsalgorithmus

Minimierungsalgorithmus

- Tabelle aller Zustandspaare $\{q, q'\}$, $q \neq q'$ erstellen
- Markiere alle Paare $\{q, q'\}$ mit $q \in F$ und $q' \notin F$ (oder umgekehrt)
- Wiederhole solange neue Markierungen hinzukommen

Der Minimierungsalgorithmus (The Table Filling Algorithm)



<https://youtu.be/2FNTDtabQz4>
(<https://youtu.be/nafVvug40vo>)

- Für jedes unmarkierte Paar $\{q, q'\}$ und jeden Buchstaben $a \in \Sigma$
 - * Wenn $\{\delta(q, a), \delta(q', a)\}$ markiert, markiere $\{q, q'\}$
- Verschmelze unmarkierte Zustandspaare

2

Kontextfreie Sprachen und Kellerautomaten

Wenn Sie eine Programmiersprache wie Java, C oder ein Format wie JSON beschreiben wollen, kommen Sie mit regulären Sprachen nicht mehr aus.

Selbst einfache Sprachen bei denen nur die Anzahl zweier Buchstaben gleich sein soll, sind bereits nicht regulär. Sie können nicht durch reguläre Ausdrücke beschrieben oder endliche Automaten erkannt werden. Korrekte Klammerungen und arithmetische Ausdrücke können ebenfalls nicht von endlichen Automaten geprüft werden. Programmiersprachen wie C und Java verlangen insbesondere korrekte Klammerungen von Blöcken und können deshalb nicht regulär sein. Für die Beschreibung solcher Sprachen benötigen wir einen ausdrucksstärkeren Formalismus. Hier benutzt man (kontextfreie) *Grammatiken*. Jede aktuelle Programmiersprache ist durch eine solche Grammatik definiert aber auch an vielen anderen Stellen tauchen Grammatiken auf, z.B. bei der Definition von JSON.

Grammatiken enthalten eine Menge von Ersetzungsregeln. Entsprechend dieser Regel können wir von einem Startsymbol ausgehend durch textuelle Ersetzungen Wörter bilden.

Wir beschreiben zuerst allgemein Grammatiken und klassifizieren sie danach anhand Ihrer erlaubten Regeln und damit verbundenen Ausdrucksstärke in der Chomsky-Hierarchie. Wir betrachten danach besonders die kontextfreien Grammatiken, die in der Lage sind Klammerstrukturen beschreiben und auch für die Beschreibung von Programmiersprachen und im Compilerbau verwendet werden. Zu diesen Sprachen gibt es ein äquivalentes Automatenmodell, die Kellerautomaten.

2.1 Grammatiken - Idee

Grammatiken sind Ersetzungssysteme. Die Grammatik gibt ein Startsymbol und Ersetzungsregeln vor. Sie unterscheidet dazu ein Alphabet von Terminalsymbolen aus denen am Ende die Wörter bestehen sollen und ein Alphabet von Nichtterminalsymbolen, die nur in Zwischenschritten vorkommen dürfen. Die Grammatik in Beispiel 48 erzeugt die nicht reguläre Sprache $a^n b^n$. Die Regel $S \rightarrow aSb \mid \varepsilon$ ist dabei so zu verstehen, dass das Symbol S entweder durch das Wort aSb ersetzt werden kann oder durch das leere Wort. Auf diese Weise können

Grammatiken und Ableitungen
(Grammars and Derivations)



<https://youtu.be/Yu4c5oFr4w4>
(https://youtu.be/D6s6o_yv0Us)

Wörter wie in Beispiel 49 abgeleitet werden.

Definition 47: Grammatik und Konventionen für Symbole

Eine Grammatik G ist eine Struktur $G = (N, \Sigma, P, S)$ wobei

- N ein Alphabet von Nichtterminalsymbolen ist,
- Σ ein Alphabet von Terminalsymbolen ist,
- P eine Menge von Produktionen oder Regeln der Form $\alpha \rightarrow \beta$ mit ist,
- $S \in N$ das Startsymbol ist.

Die Art der erlaubten Produktionen bestimmt die Klasse der Grammatik und damit deren Ausdrucksmächtigkeit. Die Klassen werden im nächsten Abschnitt definiert. Üblicherweise verwendet man folgende Konventionen:

- Großbuchstaben (A, B, C, D) für Nichtterminale
- Kleinbuchstaben (a, b, c, d) für Terminale
- Griechische Kleinbuchstaben (α, β, γ) für Wörter aus Terminal- und Nichtterminalsymbolen
- Kleinbuchstaben am Ende des Alphabets u, v, w, x, y, z für Wörter aus Terminalsymbolen
- In Produktionen verwendet man $|$ für Alternativen $S \rightarrow aSb \mid \varepsilon$ anstelle von zwei Produktionen $S \rightarrow S \rightarrow aSb, S \rightarrow \varepsilon$.

Eine Grammatik definiert eine Sprache als die Menge aller Wörter, die nur Symbole aus dem Terminalalphabet enthalten und durch Anwendung der Regeln aus dem Startsymbol abgeleitet werden können.

Intuitiv ist ein Ableitungsschritt einfach die Anwendung einer Regel, wobei ein Vorkommen der rechten Seite einer Regel durch ein Vorkommen

Definition 50: Direkte Ableitung

Sei $G = (N, \Sigma, P, S)$ eine Grammatik, $\alpha_1, \alpha_2 \in (N \cup \Sigma)^*$. Dann gilt

$$\alpha_1 \beta \alpha_2 \Rightarrow_G \alpha_1 \gamma \alpha_2 \text{ genau dann wenn } \beta \rightarrow \gamma \in P$$

($\Rightarrow_G$ ist eine Relation auf $(N \cup \Sigma)^*$)

Wenn von α in endlich vielen Schritten – auch Null Schritte sind erlaubt – β abgeleitet werden kann, d.h. falls gilt $\alpha \Rightarrow_G \dots \Rightarrow_G \beta$, dann schreibt man $\alpha \Rightarrow_G^* \beta$. Wenn G aus dem Kontext klar ist, schreibt man $\Rightarrow$ statt $\Rightarrow_G$.

Beispiel 48: Grammatik für $a^n b^n$

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow aSb \mid \varepsilon\}$$

Beispiel 49: Ableitung

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaaaSbbb \\ &\Rightarrow aaaaaSbbbb \\ &\Rightarrow aaaaabbbbb \end{aligned}$$

2.2 Chomsky-Hierarchie

Grammatiken kann man da nach unterscheiden, welche Arten von Regeln man für die Produktionen zulässt. Je weniger Einschränkungen

gen man für die Art der Regeln festlegt, desto mehr Sprachen können mit Hilfe der Grammatiken erzeugt werden. Mit steigender Ausdrucksmächtigkeit nimmt allerdings gleichzeitig die Analysierbarkeit ab. Die Chomsky-Hierarchie bietet eine solche Klassifikation.

Wir beginnen mit der allgemeinsten Klasse, bei der nur gefordert wird, dass auf der linken Seite jeder Produktion mindestens ein Nichtterminal steht. Als Maschinen-Modell sind die Turing-Maschinen äquivalent zu diesen Grammatiken. Turing-Maschinen betrachten wir im letzten Teil.

Definition 51: CH-0 Grammatik

Eine Grammatik $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ ist vom Typ Chomsky-0 (CH-0), wenn die Produktionen die Form haben

$$\alpha \rightarrow \beta$$

wobei $\alpha \in (N \cup \Sigma)^* N (N \cup \Sigma)^*$, $\beta \in (N \cup \Sigma)^*$. Eine Sprache L ist Chomsky-0 genau dann wenn es eine Chomsky-0 Grammatik G mit $L(G) = L$ gibt.

Die kontextsensitiven Grammatiken erlauben, ein Nichtterminal in einem festen "Kontext" (Zeichenkette links/rechts vom Nichtterminal) durch eine nichtleere Zeichenkette zu setzen, wobei der Kontext erhalten bleiben muss. Zu diesen Grammatiken sind die linear beschränkten Automaten äquivalent. Diese Grammatiken und Automaten werden hier nicht weiter betrachtet.

Definition 52: CH-1 Grammatiken

Eine Grammatik $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ ist vom Typ Chomsky-1 (CH-1) oder kontextsensitiv wenn die Produktionen die Form haben

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$$

wobei $A \in N$, $\alpha_1, \alpha_2 \in (N \cup \Sigma)^*$, $\beta \in (N \cup \Sigma)^+$. Zusätzlich ist $S \rightarrow \varepsilon$ erlaubt, allerdings darf S dann nicht auf der rechten Seite einer Produktion vorkommen. Eine Sprache L ist Chomsky-1 oder kontextsensitiv genau dann wenn es eine Chomsky-1 Grammatik G mit $L(G) = L$ gibt.

Besonders bedeutsam sind kontextfreie Grammatiken, da sie zur Definition von Programmiersprachen eingesetzt werden können. Bei diesen Grammatiken darf auf der linken Seite einer Produktion nur genau ein Nichtterminal vorkommen. Mit Hilfe solcher Grammatiken kann man Klammerungen, Arithmetische Ausdrücke, Palindrome und vieles andere beschreiben. Äquivalent zu diesen Grammatiken sind die Kellerautomaten, die wir am Ende des Teils betrachten.

Definition 53: CH-2 Grammatiken

Eine Grammatik $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ ist vom Typ Chomsky-2 oder kontextfrei, wenn die Produktionen die Form haben

$$A \rightarrow \beta$$

wobei $A \in N$, $\beta \in (N \cup \Sigma)^*$. Eine Sprache L ist Chomsky-2 oder

Die Chomsky-Hierarchie (The Chomsky-Hierarchy)



<https://youtu.be/lTVR-lNbnfY>
(<https://youtu.be/xIw3KFAQKj8>)

Ableitungsbäume und Mehrdeutigkeit (Parse Trees and Ambiguity)



<https://youtu.be/hbpcANpnuVs>
(<https://youtu.be/ZsTITWppouk>)

Spezielle kontextfreie Grammatiken (Special Contextfree Grammars)



https://youtu.be/th_9_Muas3o
(<https://youtu.be/nxqhcuuNnW0>)

kontextfrei genau dann wenn es eine Chomsky-2 Grammatik G mit $L(G) = L$ gibt.

Die restriktivste Form sind die rechtslinearen Grammatiken, bei denen auf der rechten Seite nur maximal ein Nichtterminal vorkommen darf, das dann auch ganz rechts steht. Mit solchen Grammatiken können reguläre Sprachen beschrieben werden.

Definition 54: CH-3 Grammatiken

ine Grammatik $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ ist vom Typ Chomsky-3 der rechtslinear wenn die Produktionen die Form haben

$$A \rightarrow aB, A \rightarrow a \text{ oder } A \rightarrow \varepsilon$$

wobei $A, B \in N, a \in \Sigma$. Eine Sprache L ist Chomsky-3 oder regulär genau dann wenn es eine Chomsky-3 Grammatik G mit $L(G) = L$ gibt.

Den Zusammenhang zwischen regulären Sprachen und Rechtslinearen Grammatiken betrachte wir genauer und zeigen, wie zu jedem endlichen Automaten eine äquivalente Grammatik konstruiert werden kann und umgekehrt zu jeder rechtslinearen Grammatik ein endlicher Automat. Die Grundidee dabei ist, dass das in der Ableitung immer ganz rechts stehende Nichtterminal dem Zustand eines Automaten entspricht.

Theorem 55: CH-3 = Regulär

Eine Sprache ist regulär genau dann, wenn sie Chomsky-3 ist.

Beweis 56: CH-3 = Regulär

$\Rightarrow$ -Richtung: Sei L regulär. Dann gibt es einen DFA $A = (Q, \Sigma, \delta, q_0, F)$, der L akzeptiert. Wir konstruieren eine CH-3 Grammatik $G = (Q, \Sigma, P, q_0)$.

- Für jede Transition $q \xrightarrow{a} q'$ für Produktion $q \rightarrow aq'$ zu P hinzu.
- Für jeden akzeptierenden Zustand q_f füge Produktion $q_f \rightarrow \varepsilon$ zu P hinzu.

Dann gilt $L = L(G)$.

$\Leftarrow$ -Richtung: Sei $G = (N, \Sigma, P, S)$ eine CH-3 Grammatik. Konstruiere NEA $A = (N \cup \{q_f\}, \Sigma, \delta, \{S\}, F)$, der $L(G)$ akzeptiert wie folgt:

- Für jede Produktion $A \rightarrow aB$ füge Transition $A \xrightarrow{a} B$ hinzu.
- Für jede Produktion $A \rightarrow a$ füge Transition $A \xrightarrow{a} q_f$ hinzu.
- Mache q_f akzeptierend.
- Für jede Produktion $A \rightarrow \varepsilon$ mache A akzeptierend.

Regulär = Chomsky-3 (Regular = Chomsky-3)



<https://youtu.be/U0Kz8KE93qc>
(<https://youtu.be/VyMmpv5jURo>)

2.3 Elimination von Epsilon-Produktionen

Produktionen, die ein Nichtterminal durch das leere Wort ersetzen können störend sein. In einer Ableitung kann dadurch die Satzform wieder kürzer werden. Wir zeigen, dass die Produktionen zwar praktisch zur Definition von Grammatiken sind, aber algorithmisch eliminiert werden können.

Definition 57: ε -Produktion

Eine Produktion der Form $A \rightarrow \varepsilon$ heißt Epsilon-Produktion.

Im ersten Schritt finden wir alle Nichtterminale aus denen in beliebig vielen Schritten das leere Wort abgeleitet werden kann. Dazu beginnen wir mit den Nichtterminalen, die schon direkt ε -Produktionen haben und erweitern die Menge dann schrittweise bis die Menge stabil ist.

Algorithmus: Finden aller X mit $X \Rightarrow^* \varepsilon$

1. Finden aller Nichtterminale X mit Produktionen $X \rightarrow \varepsilon$.
2. Wenn es eine Produktion $X \rightarrow X_1 \dots X_n$ gibt so dass für schon $X_i \Rightarrow^* \varepsilon$ für alle $1 \leq i \leq n$ bekannt ist, dann gilt auch $X \Rightarrow^* \varepsilon$.
3. Auf diese Weise werden schrittweise alle Nichtterminale mit $X \Rightarrow^* \varepsilon$ in einer Menge aufgesammelt, bis sich die Menge nicht mehr ändert.

Algorithmus: Elimination von ε -Produktionen

- Für jedes Vorkommen eines Nichtterminals X mit $X \Rightarrow^* \varepsilon$ auf einer rechten Seite einer Produktion wird eine *neue* Regel hinzugefügt bei der X auf der rechten Seite entfernt ist. Die ε -Produktionen werden quasi vorweggenommen. Erzeuge in dem Schritt keine neuen ε -Produktionen.
- Entferne alle ε -Produktionen.
- Falls $S \Rightarrow^* \varepsilon$ füge frisches Startsymbol S' und Produktion $S' \rightarrow S \mid \varepsilon$ hinzu.

Theorem 58: Entscheidbarkeit des Wortproblems für kontextfreie Sprachen

Das Wortproblem für kontextfreie Sprachen ist entscheidbar.

Algorithmus: Entscheidbarkeit des Wortproblems für kontextfreie Sprachen

1. Eliminiere alle ε Produktionen.
2. Erzeuge alle Ableitungen
 - (a) Backtracking falls Satzform schon geprüft wurde
 - (b) Backtracking falls Satzform länger als w ist
 - (c) Gib true zurück falls w gefunden ist

Elimination von ε -Produktion (Eliminating ε Productions)



<https://youtu.be/HZNwKiJIMPU>
(<https://youtu.be/j9KDazY4ALU>)

3. Gib false zurück, falls alle Satzformen ohne Erfolg geprüft wurden.

2.4 Abschlusseigenschaften Kontextfreier Sprachen

- Ein JSON-Object enthält beliebig viele MEMBER durch Komma getrennt. Das entspricht dem Sternabschluss.
- Jeder MEMBER besteht aus einem STRING gefolgt von einem Doppelpunkt und einem ELEMENT. Es handelt sich um Konkatination.
- Ein ELEMENT kann ein OBJECT, STRING, NUMBER, ARRAY (oder anderes) sein. Hier haben wir Alternative / Vereinigung.

JSON - Beispiel

```
{
  "name": "John Smith",
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ]
}
```

Die JavaScript Object Notation (JSON) ist über eine Grammatik definiert. An diesem Beispiel erkennt man typische Anforderungen, die über eine Grammatik spezifiziert werden soll. In den folgenden Theoremen zeigen wir, dass und wie diese Operationen auf Grammatikebene umgesetzt werden können. Die Konstruktionen finden sich jeweils in den Beweisen.

Theorem 59: Abschluss unter Vereinigung

Die kontextfreien Sprachen sind unter Vereinigung abgeschlossen.

Beweis 60: Abschluss unter Vereinigung

Seien $G_1 = (N_1, \Sigma, P_1, S_1)$ und $G_2 = (N_2, \Sigma, P_2, S_2)$ kontextfreie Grammatiken mit $N_1 \cap N_2 = \emptyset$. Sei weiter S ein frisches Nichtterminal, das nicht in N_1 und nicht in N_2 vorkommt. Dann erzeugt die kontextfreie Grammatik

$$G_V = (N_1 \cup N_2 \cup \{S\}, \Sigma, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}, S)$$

die Sprache $L(G_1) \cup L(G_2)$.

Vereinigung, Stern (Union, Star)
Konkatenation, Stern (Concatenation, Star)



https://youtu.be/aXn1x_Bo1_4
(<https://youtu.be/AXy65rWPVcw>)

Theorem 61: Abschluss unter Konkatenation

Die kontextfreien Sprachen sind unter Vereinigung abgeschlossen.

Beweis 62: Abschluss unter Konkatenation

Seien $G_1 = (N_1, \Sigma, P_1, S_1)$ und $G_2 = (N_2, \Sigma, P_2, S_2)$ kontextfreie Grammatiken mit $N_1 \cap N_2 = \emptyset$. Sei weiter S ein frisches Nichtterminal, dass nicht in N_1 und nicht in N_2 vorkommt. Dann erzeugt die kontextfreie Grammatik

$$G_C = (N_1 \cup N_2 \cup \{S\}, \Sigma, P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}, S)$$

die Sprache $L(G_1) \circ L(G_2)$.

Theorem 63: Abschluss unter Stern

Die kontextfreien Sprachen sind unter Stern abgeschlossen.

Beweis 64: Abschluss unter Stern

Sei $G_1 = (N_1, \Sigma, P_1, S_1)$ eine kontextfreie Grammatik. Sei weiter S ein frisches Nichtterminal, dass nicht in N_1 vorkommt. Dann erzeugt die kontextfreie Grammatik

$$G_S = (N_1 \cup \{S\}, \Sigma, P_1 \cup \{S \rightarrow SS_1 \mid \varepsilon\}, S)$$

die Sprache $L(G_1)^*$.

2.5 Das Pumping-Lemma für kontextfreie Sprachen

Das Pumping-Lemma für reguläre Sprachen beschreibt eine Eigenschaft, die alle regulären Sprachen haben müssen. Wenn wir zeigen können, dass eine Sprache diese Eigenschaft nicht besitzt, können wir folgern, dass die Sprache nicht regulär ist. Die Grundidee bei regulären Sprachen war, dass wenn das Wort nur lang genug ist, der endliche Automat einen Zyklus durchlaufen muss und dieser Zyklus beliebig oft wiederholt werden kann.

Diese Idee kann man auf kontextfreie Sprachen übertragen. Wenn das Wort nur lang genug ist, muss in der Ableitung mindestens ein Nichtterminal doppelt verwendet werden. Da die Grammatik kontextfrei ist, kann diese Teilableitung dann aber auch beliebig oft eingesetzt werden.

Lemma 65: Pumping Lemma für kontextfreie Sprachen

Sei L eine kontextfreie Sprache. Dann gibt es ein $p \in \mathbb{N}$, so dass für alle $z \in L$ mit $|z| \geq p$ eine Zerlegung $z = uvwxy$ existiert mit

1. $|vwx| \leq p$ (Mittelstück hat maximal Länge von p)
2. $|vx| \geq 1$ (also höchstens eins von v und x ist leer)
3. $uv^iwx^iy \in L$ für alle $i \in \mathbb{N}$

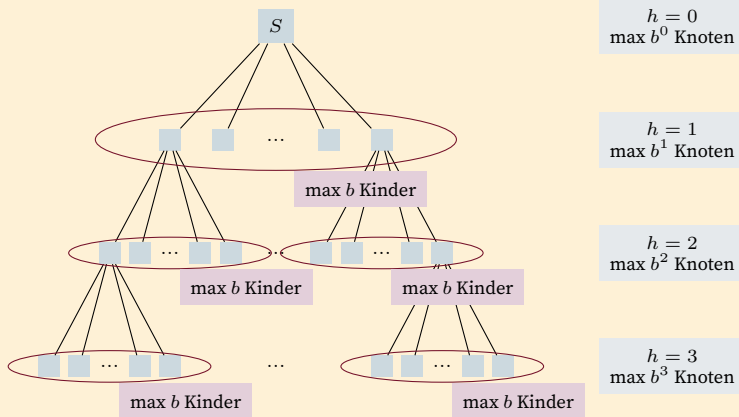
Das Pumping-Lemma für kontextfreie Sprachen (*The Pumping-Lemma for Contextfree Languages*)



<https://youtu.be/Y7zhIEaaTcs>
(<https://youtu.be/WXChgn8VsKw>)

Beweis 66: Pumping-Lemma für für kontextfreie Sprachen

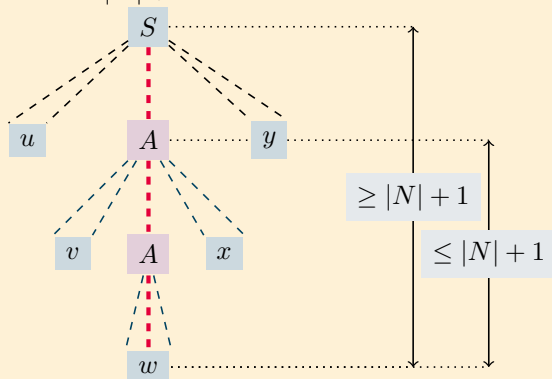
Sei L eine kontextfreie Sprache und $G = (N, \Sigma, P, S)$ eine zugehörige kontextfreie Grammatik ($L(G) = L$) und sei b die maximale Anzahl an Symbolen, die auf einer rechten Seite einer Produktion in P vorkommt. Dieses b ist die maximale Anzahl an Kindern, die ein Syntaxbaum eines Wortes aus $L(G)$ haben kann.



Damit kann ein Syntaxbaum der Höhe h maximal b^h Blätter haben. Die Blätter sind genau die Buchstaben des zugehörigen Wortes. Also kann ein Wort mit einem Syntaxbaum der Höhe h maximal b^h Zeichen besitzen. Umgekehrt bedeutet das, dass ein Syntaxbaum für ein Wort der Länge b^h mindestens eine Höhe von h besitzen muss. Wir definieren eine Konstante

$$p = b^{|N|+1}.$$

Ein Wort $z \in L(G)$ mit mindestens p Zeichen ($|z| \geq p$) muss damit einen Syntaxbaum mit einer Höhe von mindestens $|N| + 1$ besitzen. Da die Grammatik nicht eindeutig sein muss, kann es mehrere Syntaxbäume für das gleiche Wort geben. Wir wählen den kleinsten Syntaxbaum. Auch dieser Baum muss noch eine Höhe von mindestens $|N| + 1$ besitzen.

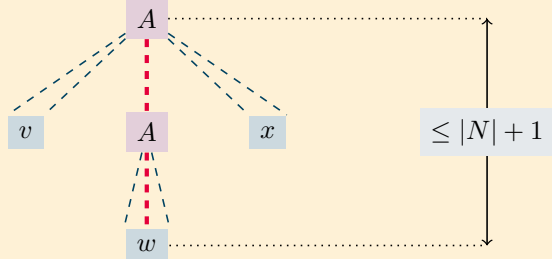


Es gibt also einen Weg mit mindestens $|N| + 2$ Knoten – die Wurzel liegt auf Höhe $h = 0$, deshalb hat der Weg einen Knoten mehr. Von diesen Knoten sind mindestens $|N| + 1$ Knoten Nichtterminale und ein Knoten – das Blatt – ist ein Terminal. Da dieser Pfad mit $|N| + 1$ Nichtterminalen mehr Nichtterminale als die Grammatik enthält, muss ein Nichtterminal A doppelt auftauchen. Da-

mit muss der Syntaxbaum die dargestellte Form besitzen, wobei u, v, w, x, y die Konkatenation der Blätter der jeweiligen Teilbäume sind. Es sind also Wörter aus Terminalen und es gilt $z = uvwxy$. Es gibt also eine Ableitung

$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvwxy = z.$$

Wir betrachten das am tiefsten im Baum vorkommende doppelte Paar. Diese Paar liegt in einem Teilbaum mit maximaler Höhe $|N| + 1$, da vom Blatt angefangen auf dieser Höhe ein Nichtterminal schon doppelt vorkommen muss.



Dieser Teilbaum kann nach vorheriger Überlegung nur maximal $b^{|N|+1} = p$ Blätter haben. Damit folgt

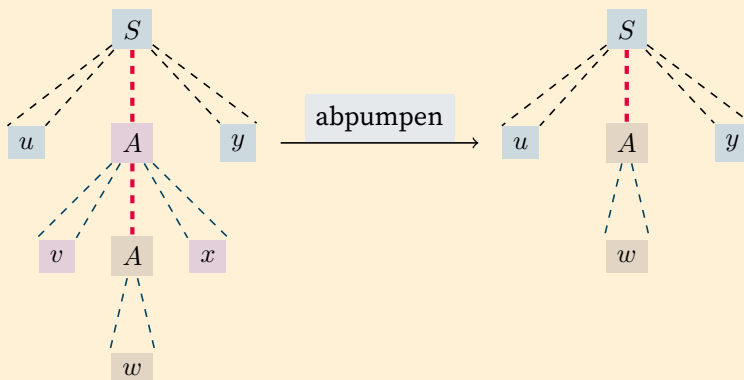
$$|vwx| \leq p.$$

Da der Baum minimal für das Wort z ist, können nicht sowohl v als auch x das leere Wort sein. Andernfalls könnte man die Schritte $A \Rightarrow^* vAx$ nämlich weglassen und würde einen kleinen Baum bekommen. Wir mussten den kleinsten Baum also voraussetzen, da die Grammatik Produktionen wie $A \rightarrow A$ enthalten könnte. Es gilt also

$$|vx| \geq 1.$$

Wir können den Teilbaum zu $A \Rightarrow^* vAx$ entfernen – abpumpen – und erhalten die Ableitung

$$S \Rightarrow^* uAy \Rightarrow^* uwx.$$



Wir können den Teilbaum zu $A \Rightarrow^* vAx$ auch ein weiteres Mal einfügen – aufpumpen – und erhalten die Ableitung

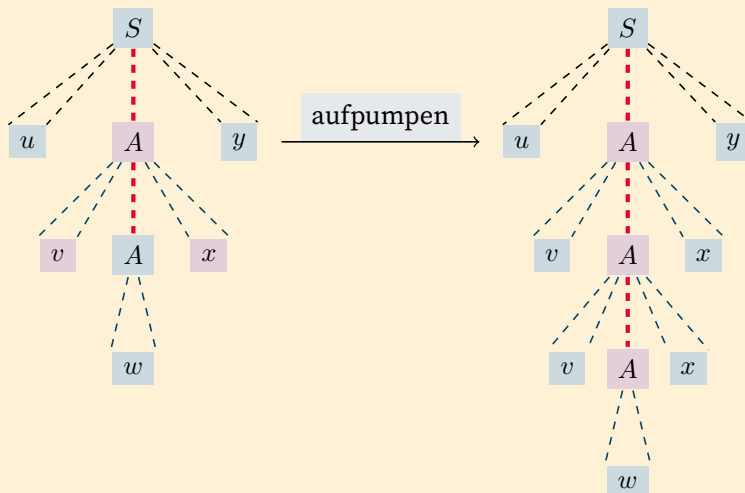
$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvvAxy \Rightarrow^* uvvwxy = uv^2wx^2y.$$

Aufpumpen ist mehrfach möglich, d.h. für alle $i \geq 1$ gilt

$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uv^iwx^iy.$$

Da wir den Sonderfall $i = 0$ schon betrachtet haben folgt

$$uv^iwx^iy \in L(G) \text{ für alle } i \geq 0.$$



Theorem 67: $a^n b^n c^n$ ist nicht kontextfrei

Die Sprache $L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht kontextfrei.

Beweis 68: $a^n b^n c^n$ ist nicht kontextfrei

Wir nehmen an, dass L kontextfrei ist. Dann gibt es nach Pumping-Lemma eine Konstante p . Wir betrachten das Wort $z = a^p b^p c^p$. Wegen $z \in L$ und $|z| \geq p$ hat z eine Zerlegung $z = uvwxy$ nach Pumping-Lemma. Wegen $|vwx| \leq p$ kann vwx nicht sowohl as als auch cs enthalten. Es muss also einen der Buchstabentypen nicht enthalten.

1. Fall vwx enthält keine a Wir pumpen ab und betrachten uwy . Da $vx \neq \varepsilon$ hat uwy jetzt zu viele a , da bs oder cs abgepumpt wurden aber keine as .
2. Fall vwx enthält keine c Wir pumpen ab und betrachten uwy . Da $vx \neq \varepsilon$ hat uwy jetzt zu viele c , da as oder bs abgepumpt wurden aber keine cs .

In jedem der Fälle liegt uwy nicht in L im Widerspruch zum Pumping-Lemma, also ist L nicht kontextfrei.

$a^n b^n c^n$ ist nicht kontextfrei
($a^n b^n c^n$ is not contextfree)



https://youtu.be/zr_8A2atl3g
(https://youtu.be/u83cS9rhd_Y)

2.6 Schnitt und Komplement

Die regulären Sprachen sind unter Schnitt und Komplement abgeschlossen, die kontextfreien Sprachen nicht.

Theorem 69: Schnitt und Komplement

Die kontextfreien Sprachen sind nicht unter Schnitt und Komplement abgeschlossen.

Beweis 70: Schnitt und Komplement

Die Sprachen

$$L_1 = \{a^n b^n c^m \mid n, m \in \mathbb{N}\} \text{ und}$$

$$L_2 = \{a^n b^m c^m \mid n, m \in \mathbb{N}\}$$

sind kontextfrei, da sie von den kontextfreien Grammatiken

$$G_1 = (\{S, A, B\}, \{a, b, c\}, \{S \rightarrow AB, A \rightarrow aAb \mid \varepsilon, B \rightarrow cB \mid \varepsilon\}, S)$$

$$G_2 = (\{S, A, B\}, \{a, b, c\}, \{S \rightarrow AB, A \rightarrow aA \mid \varepsilon, B \rightarrow bBc \mid \varepsilon\}, S)$$

erzeugt werden. Der Schnitt $L_1 \cap L_2 = \{a^n b^n c^n\}$ ist jedoch nicht kontextfrei wie oben mit Hilfe des Pumping-Lemmas gezeigt. Also sind die kontextfreien Sprachen nicht gegen Schnitt abgeschlossen.

Da die kontextfreien Sprachen gegen Vereinigung aber nicht gegen Schnitt abgeschlossen sind, können sie auch nicht gegen Komplement abgeschlossen sein, weil die Beziehung

$$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$$

gilt.

Schnitt und Komplement (Intersection and Complement)

<https://youtu.be/DUnYETMo9bE>
(<https://youtu.be/m2fLB49z1wI>)

2.7 Kellerautomaten

Intuitiv können endliche Automaten Sprachen wie $a^n b^n$ nicht akzeptieren, weil Sie über keinen externen Speicher verfügen, in dem sie sich etwas merken können. Endliche Automaten müssen sich alles in den Zuständen merken.

Wir erweitern nun unser Automatenmodell und führen einen Stack als zusätzlichen Speicher ein. Bei einem Stack hat man immer nur Zugriff auf das oberste Element des Stapels, man kann ein Element vom Stack runterholen (pop) ein Element auf den Stack legen (push).

Wir betrachten in diesem Teil nur nichtdeterministische Kellerautomaten. Sie holen in jedem Rechenschritt ein Zeichen vom Stack. Sie können ein Zeichen aus der Eingabe lesen oder eine sogenannte ε -Transition nehmen, die nicht aus der Eingabe liest. Abhängig von der Eingabe und dem runtergeholten Zeichen vom Stack kann der Kellerautomat, eine Transition nehmen, des Zustand wechseln und neue Zeichen auf den Stack schreiben.

Formal sind Kellerautomaten 7-Tupel. Damit auf dem Keller Zeichen vorkommen können, die nicht in der Eingabe vorkommen nutzen wir ein separates Kelleralphabet. Außerdem spezifizieren wir einen initialen Kellerinhalt.

Kellerautomaten (Pushdown Automata)

<https://youtu.be/faYULCjpBtw>
(<https://youtu.be/3ECg6F9ssTA>)

Die Überföhrungsfunktion liefert in Abhängigkeit vom aktuellen Zustand, einem Eingabezeichen oder ε – für Übergänge, die nicht aus der Eingabe lesen sollen – und dem aktuellen Zeichen ganz oben auf dem Stack, eine Menge von Kombinationen von Nachfolgezustand und Elementen, die auf den Stack zu legen sind.

Definition 71: Kellerautomat (pushdown automaton (PDA))

Ein nichtdeterministischer Kellerautomat ist ein Tupel $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.

- Q ist eine endliche Menge von **Zuständen** (states)
- Σ ist eine endliche Menge, das **Eingabealphabet** (input alphabet)
- Γ ist eine endliche Menge, das **Kelleralphabet** (stack alphabet)
- $\delta : (Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma) \rightarrow \mathcal{P}(Q \times \Gamma^*)$ ist die **Überföhrungsfunktion** (transition function)
- $q_0 \in Q$ ist der **Anfangszustand** (start / initial state)
- Z_0 ist das oberste Kellersymbol beim Start
- $F \subseteq Q$ ist die Menge der **Endzustände** (final / accepting states)

Das Verhalten des Kellerautomaten hängt vom Zustand, der restlichen Eingabe und dem Kellerinhalt ab. Diese drei Teile fassen wir als Konfiguration zusammen. Bei den endlichen Automaten haben wir uns nur den Zustand gemerkt.

Definition 72: Konfiguration (instantaneous description)

Eine Konfiguration $(q, a_1 \dots a_n, A_1 \dots A_k)$ ist ein Tripel aus $Q \times \Sigma^* \times \Gamma^*$ aktuellem Zustand, restlichem Eingabewort und Kellerinhalt.

Die oben beschriebene Idee eines Rechenschritts wird nun formalisiert. Wir unterscheiden dabei Schritte, die ein Zeichen aus der Eingabe lesen und Schritte, die das nicht tun. Wichtig ist, dass aus der Definition direkt folgt, dass ein Kellerautomat nach dieser Definition mit einem leeren Keller keine Schritte mehr machen kann.

Definition 73: Rechenschritt

Für einen Kellerautomaten P ist $\vdash$ definiert durch

$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_2 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{gdw. } \delta(q, a_1, A_1) \ni (q', B_1 \dots B_m)$$

und

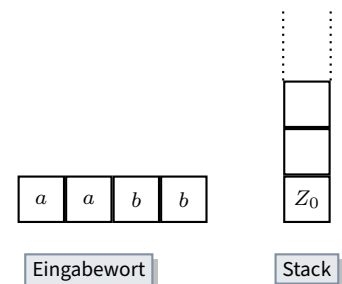
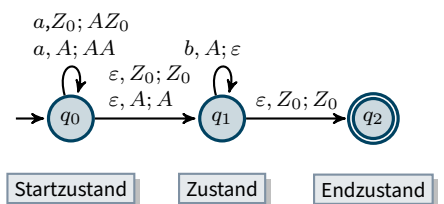
$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_1 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{gdw. } \delta(q, \varepsilon, A_1) \ni (q', B_1 \dots B_m)$$

Wir schreiben $(q, w, \alpha) \vdash^* (q', w', \alpha')$ falls die Konfiguration (q', w', α') in endlich vielen Schritten aus (q, w, α) erreichbar ist.

Kellerautomaten (formale Definition) (Pushdown Automata (formal definition))



<https://youtu.be/5IzTX0ZuJu4>
(<https://youtu.be/sSVJYIuxKNA>)



Es gibt zwei übliche Arten, einen Kellerautomaten ein Wort akzeptieren zu lassen. Die erste Möglichkeit ist über Endzustände wie bei endlichen Automaten. Wenn es eine Folge von Rechenschritten gibt, die das Eingabewort vollständig liest und in einen Endzustand führt, wird das Wort akzeptiert.

Definition 74: Akzeptanz mit Endzustand

Sei $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ ein Kellerautomat. P akzeptiert ein Wort $w \in \Sigma^*$ mit Endzustand falls $(q_0, w, Z_0) \vdash^* (q_f, \epsilon, \alpha)$ und $q_f \in F$, also falls es die Möglichkeit gibt, dass der Kellerautomat das ganze Eingabewort vollständig abarbeitet und sich dann in einem Endzustand befindet.

Die mit Endzustand akzeptierte Sprache ist

$$L(P) = \{w \in \Sigma^* \mid P \text{ akzeptiert } w \text{ mit Endzustand}\}.$$

Alternativ kann ein Kellerautomat auch dadurch akzeptieren, dass der Keller leer wird. Man kann zeigen, dass sich beide Arten ineinander konvertieren lassen. Für beide Richtungen der Konvertierung führt man einen frischen Bodenmarker ein, der direkt am Anfang in den Keller gelegt wird. Darauf wird der ursprüngliche initiale Kellereintrag gepackt.

Ein Kellerautomat, der mit Endzustand akzeptiert kann dann einfach in allen Endzuständen den Keller leeren. Aufgrund des neuen Bodenmarkers kann ein leerer Keller in anderen Zuständen nicht vorkommen. So wird aus einem Kellerautomaten, der mit Endzustand akzeptiert ein Kellerautomat, der mit leerem Keller akzeptiert.

Ein Kellerautomat, der mit leerem Keller akzeptiert, kann einfach von jedem Zustand aus ϵ -Transitionen hinzufügen, die in einen Endzustand führen, sobald der neue Bodenmarker gefunden wird. So wird aus einem Kellerautomat, der mit leerem Keller akzeptiert ein Automaten, der mit Endzustand akzeptiert.

Definition 75: Akzeptanz mit leerem Keller

Sei $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ ein Kellerautomat. P akzeptiert ein Wort $w \in \Sigma^*$ mit leerem Keller falls $(q_0, w, Z_0) \vdash^* (q', \epsilon, \epsilon)$, also falls der Kellerautomat bei Abarbeiten des Wortes den Keller leeren kann. Beachte, dass ein Kellerautomat mit leerem Keller keine Schritte machen kann. Die mit leerem Keller akzeptierte Sprache ist dann

$$N(P) = \{w \in \Sigma^* \mid P \text{ akzeptiert } w \text{ mit leerem Keller}\}.$$

Kellerautomat für wcw^R (A PDA for wcw^R)



<https://youtu.be/-oNk5wvmPnk>
(<https://youtu.be/-DG90BMN90Y>)

Kellerautomat für Palindrome gerader Länge (A PDA for Palindromes of even length)



https://youtu.be/rJp5rE4_qzI
(<https://youtu.be/LnKX6UnptLI>)

2.8 Kellerautomaten und kontextfreie Sprachen

Die Kellerautomaten sind das zu den kontextfreien Sprachen gehörende Maschinenmodell. Zu jeder kontextfreien Grammatik kann man einen Kellerautomaten bauen, der die gleiche Sprache akzeptiert und umgekehrt.

Für eine gegebene Grammatik kann sehr einfach ein Kellerauto-

mat konstruiert werden. Die Grundidee ist dabei, dass der Kellerautomat die Ableitung des Eingabewortes auf dem Stack herstellt. Wenn oben auf dem Stack Terminalzeichen liegen, müssen die den Zeichen der Eingabe entsprechen und können gematcht werden. Wenn oben auf dem Stack ein Nichtterminal liegt, kann der Kellerautomat nicht-deterministisch eine Produktion auswählen und das Nichtterminal oben auf dem Keller entsprechend der Produktion ersetzen.

Der Kellerautomat hat deshalb drei Zustände.

1. Zuerst wird auf den Bodenmarker $\#$ das Startsymbol der Grammatik gelegt.
2. Im zweiten Zustand wird entweder ein Zeichen der Eingabe gematcht, sofern oben auf dem Stack ein Terminalzeichen liegt oder es wird ein Nichtterminal oben auf dem Stack entsprechend einer Produktion der Grammatik ersetzt.
3. Wenn der Bodenmarker $\#$ gefunden wird, ist der gesamte Stack gegen die Eingabe gematcht und der Kellerautomat wechselt in einen Endzustand. Falls die Eingabe komplett gelesen wurde, akzeptiert der Kellerautomat.

Theorem 76: Kellerautomaten und kontextfreie Sprachen

Zu jeder kontextfreien Grammatik G kann ein Kellerautomat P konstruiert werden, der die gleiche Sprache akzeptiert.

Beweis 77: Kellerautomaten und kontextfreie Sprachen

Sei $G = (N, \Sigma, R, S)$ eine kontextfreie Grammatik. Konstruiere einen Kellerautomaten P mit drei Zuständen. Im Keller simuliert P die Ableitung eines Wortes. Das Kelleralphabet ist deshalb die Vereinigung der Terminale und Nichtterminale zusammen mit einem frischen Bodenmarker $\#$, der sich schon initial im Keller befindet.

$$P = (\{q_0, q_1, q_F\}, \Sigma, N \cup \Sigma \cup \{\#\}, \delta, q_0, \#, \{q_F\})$$

Es gibt drei Arten von Transitionen in δ

1. An Anfang Startsymbol auf Keller legen
 $\delta(q_0, \varepsilon, \#) = \{(q_1, S\#)\}$
2. An Ende in Endzustand wechseln wenn nur noch $\#$ im Keller
 $\delta(q_1, \varepsilon, \#) = \{(q_F, \varepsilon)\}$
3. Wenn Terminal Kellertop, Eingabe matchen
 $\delta(q_1, a, a) = \{(q_1, \varepsilon)\}$ für alle $a \in \Sigma$
4. Wenn Nichtterminal Kellertop, Produktion raten und anwenden, d.h. Nichtterminal oben auf Stack durch rechte Seite einer passenden Produktion ersetzen. $\delta(q_1, \varepsilon, A) = \{(q_1, B_1 \dots B_n) \mid A \rightarrow B_1 \dots B_n \in R\}$

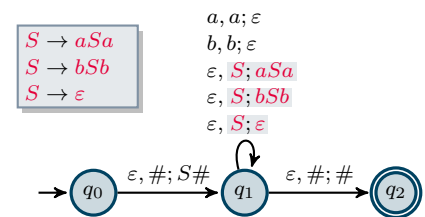


Abbildung 2.1: Grammatik und zugehöriger PDA

Grammatik $\rightsquigarrow$ Kellerautomat
(Grammar $\rightsquigarrow$ PDA)



<https://youtu.be/yXPacARHdBg>
(<https://youtu.be/SkaTpAj5DdU>)

Kellerautomaten konstruieren

Je nach Sprache findet man unter Umständen einfach eine kontextfreie Grammatik für eine Sprache. Dann ist es einfach und quasi idiotensicher aus der Grammatik den Kellerautomaten zu konstruieren anstatt den Kellerautomaten von Hand zu erstellen.

Man kann auch für jeden Kellerautomaten eine kontextfreie Grammatik bauen, die das Verhalten des Kellerautomaten simuliert. Wir geben nur diese Eigenschaft an und verzichten auf den recht technischen Beweis. Bei Interesse finden Sie den Beweis in der Standardliteratur.

Theorem 78: Kellerautomaten und kontextfreie Sprachen

Zu jedem Kellerautomaten P kann eine kontextfreie Grammatik G konstruiert werden, die die gleiche Sprache erzeugt.

3

Berechenbarkeit und Komplexität

Man könnte denken, dass Computer mit dem richtigen Programm prinzipiell jedes Problem lösen und jede Funktion berechnen können. Es zeigt sich aber, dass es nicht nur viele Probleme gibt, für die das prinzipiell nicht möglich ist. Diese Probleme sind nicht esoterisch und besondere Ausnahmefälle, sondern in der Praxis der Softwareentwicklung sogar besonders relevant.

Wenn man einmal weiß, dass ein Problem prinzipiell durch ein Programm berechnet werden kann, fragt man sich, mit welchem Aufwand diese Berechnung verbunden ist. Die wichtigen Ressourcen sind dabei Laufzeit und Speicherplatz.

3.1 Turing-Maschinen

Um untersuchen zu können, welche Probleme ein Computer mit einem Programm lösen kann, benötigen wir zuerst ein formales Modell eines Computers. Das Modell soll auf der einen Seite einfach genug sein, um sich in den Beweisen nicht in Details zu verlieren und auf der anderen Seite mächtig genug sein, um alles zu können, was ein Computer auch kann. Endliche Automaten haben keinerlei externen Speicher und müssen sich alle Informationen in ihren Zuständen merken. Wegen dieser Beschränkung können endliche Automaten selbst relativ einfache Sprachen wie die Klammersprachen nicht erkennen.

Kellerautomaten besitzen einen externen Speicher, den Stack oder Keller und können damit korrekte Klammerungen erkennen. Bei einem Stack kann jedoch immer nur auf das oberste Element zugegriffen werden.

Alan Turing hat 1936 ein einfaches Modell mit einer endlichen Kontrolle, wie bei endlichen Automaten und Kellerautomaten, und einem unendlich langen Band als Speicher vorstellt. Man kann zeigen, dass diese Turing-Maschinen alle Berechnungen durchführen können, die auch auf einem “normalen Computer” möglich sind.

Wie die anderen Automatenmodelle besitzt eine Turing-Maschine eine endliche Kontrolle, die das Verhalten steuert und ein in zwei Richtungen unendlich langes Band auf dem ein Schreib/Lese-Kopf steht. In jedem Schritt liest die Turing-Maschine das aktuelle Bandzeichen, schreibt ein Zeichen an dessen Stelle und bewegt den Kopf

Einführung in Turing-Maschinen
(Introduction to Turing-Machines)



<https://youtu.be/obwKl6y0Edg>
(https://youtu.be/_YTT_8yVZWM)

nach Links, nach Rechts oder lässt den Kopf auf der aktuellen Position.

Definition 79: Turing-Maschine

Eine deterministische Turing-Maschine (Turing Machine, TM) ist ein 7-Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ mit

- Zustandsmenge Q
- Eingabealphabet Σ
- Bandalphabet $\Gamma \supset \Sigma$
- Transitionsfunktion $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$ (L : Links, R : Rechts, N : Neutral – nicht bewegen)
- Startzustand q_0
- $\sqcup \in \Gamma$ das Blank (Leerzeichen) (Kein Eingabezeichen: $\sqcup \notin \Sigma$)
- F eine Menge von Endzuständen

Die Turing-Maschine in Abbildung 3.1 akzeptiert alle Wörter der Form $a^n b^n c^n$. Diese Sprache ist nicht kontextfrei. Die Turing-Maschine läuft dazu das Wort von links nach rechts durch und ersetzt zuerst ein a durch ein x , dann ein b und schließlich ein c . Danach wird der Kopf zurück an den Anfang bewegt. Wenn auf dem Band irgendwann nur noch x stehen, wird die Eingabe akzeptiert.

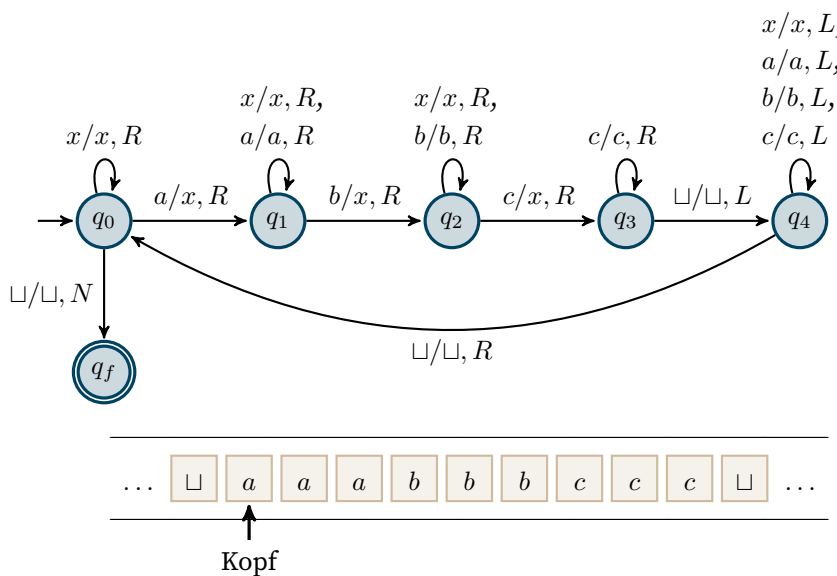


Abbildung 3.1: Turing-Maschine für $a^n b^n c^n$

beidseitig unendlich
langes Band
mit Eingabewort

Der Ablauf eines endlichen Automaten wird durch eine Folge von Zuständen beschrieben, der Ablauf eines Kellerautomaten durch eine Folge von Konfigurationen. Bei Turing-Maschinen ist das genauso. Hier muss man sich zusätzlich zum aktuellen Zustand noch Bandinhalt und Kopfposition merken.

Definition 80: Konfiguration einer Turing-Maschine

Die **Konfiguration** einer Turing-Maschine beschreibt den aktuellen Zustand q , den Bandinhalt und die Kopfposition. Dies wird kurz durch ein Wort

$$uqv$$

beschrieben. Dabei ist q der aktuelle Zustand, uw der Bandinhalt und der Kopf steht auf dem ersten Zeichen von v .

Turing-Maschinen können sowohl zum Akzeptieren von Sprachen als zum Berechnen von Funktionen verwendet. Akzeptanz von Sprachen ist ganz analog zu den anderen Automatenmodellen.

Definition 81: Akzeptierte Sprache

Eine Turing-Maschine **akzeptiert** ein Wort, wenn die Turing-Maschine angesetzt auf das Wort – d.h. das Wort ist die Eingabe – in einen Endzustand kommen kann, d.h.

$$(q_0 w) \rightarrow^* (\alpha q_f \beta), q_f \in F.$$

Die akzeptierte Sprache ist die Menge aller akzeptierten Wörter.

Da Turing-Maschinen auf das Band schreiben können, kann man damit auch eine Funktion berechnen. Der Eingabewert der Funktion steht initial auf dem Band, die Turing-Maschine wird gestartet und falls die Turing-Maschine anhält, steht der berechnete Funktionswert auf dem Band.

Definition 82: Berechnete Funktion

Eine Turing-Maschine berechnet eine Funktion, wenn sie angesetzt auf die Kodierung des Eingabewertes die Kodierung des Funktionswerts auf das Band schreibt, d.h.

$$(q_0 w) \rightarrow^* (\alpha q_f \beta), q_f \in F$$

wobei w die Codierung des Eingabewertes ist und $\alpha\beta$ die Codierung des Funktionswerts ist. Beispielscodierungen sind Binärcodierung oder Unärcodierung (Bierdeckelnotation)

Die Turing-Maschine in Abbildung 3.2 multipliziert zwei Zahlen in Unärcodierung, d.h. sie berechnet die Funktion

$$f : I^n \# I^m \mapsto I^{nm}.$$

Mehrband-Turing-Maschinen

Anstatt eines Bandes können Turing-Maschinen auch mehrere Bänder mit jeweils einem Kopf nutzen.

Diese Erweiterung des Modells erlaubt es zwar nicht, “mehr” zu akzeptieren oder zu berechnen, die Konstruktion der Turing-Maschinen wird ggf. stark vereinfacht. Die Turing-Maschine in Abbildung 3.3 kopiert zuerst Wort bis zum Zeichen c auf das zweite Band, geht dann

Turing-Maschinen zur Berechnung von Funktionen (Turing Machines Computing Functions)

https://youtu.be/PqaQt_uFAA0
(<https://youtu.be/SNnezr1kh98>)

Mehrband-Turing-Maschinen (Multi Tape Turing Machines)

<https://youtu.be/m4qI5h2rsP4>
(<https://youtu.be/5qZ9zT01Lxc>)

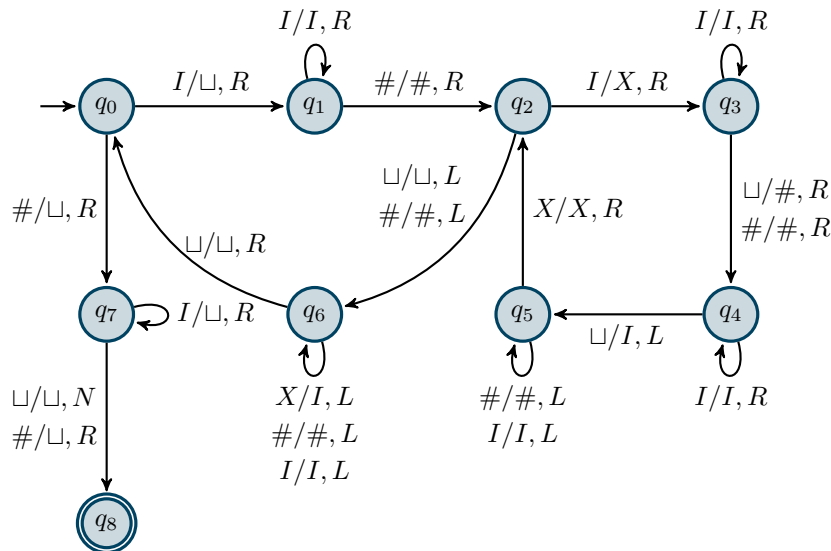


Abbildung 3.2: Multiplikation von zwei Zahlen in Unärkodierung

auf dem zweiten Band zurück und vergleicht den Inhalt von zweitem Band und den Rest des ersten Bands. In der Abbildung ist die Konfiguration zu Beginn des Matching-Schritts dargestellt.

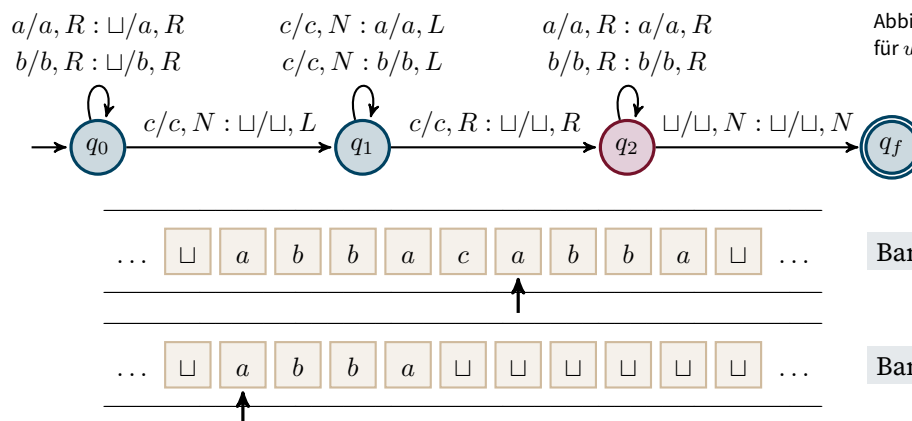


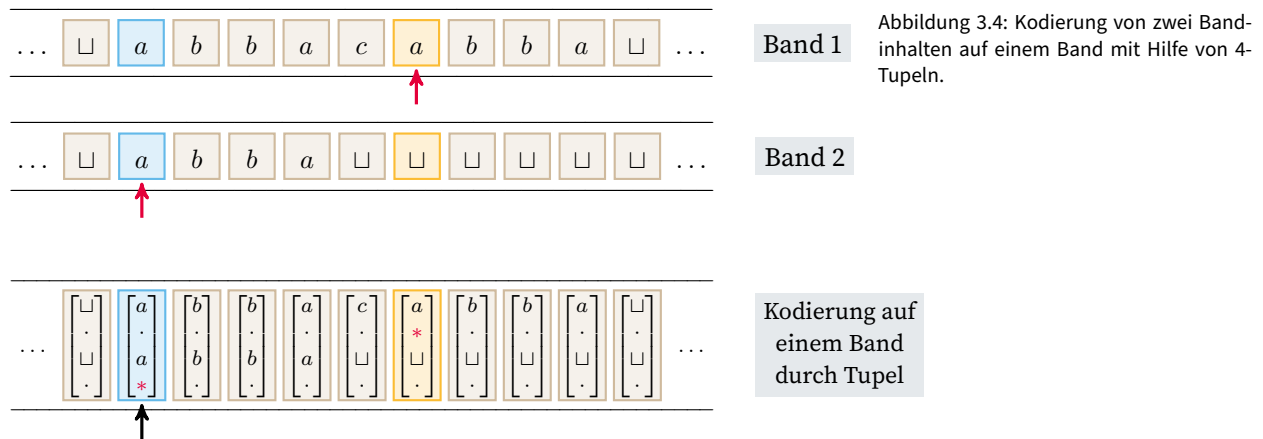
Abbildung 3.3: 2-Band-Turing-Maschine für wcw zu Beginn des Matching-Schritts

Jede Mehrband-Turing-Maschine kann durch eine Einband-Turing-Maschine simuliert werden.

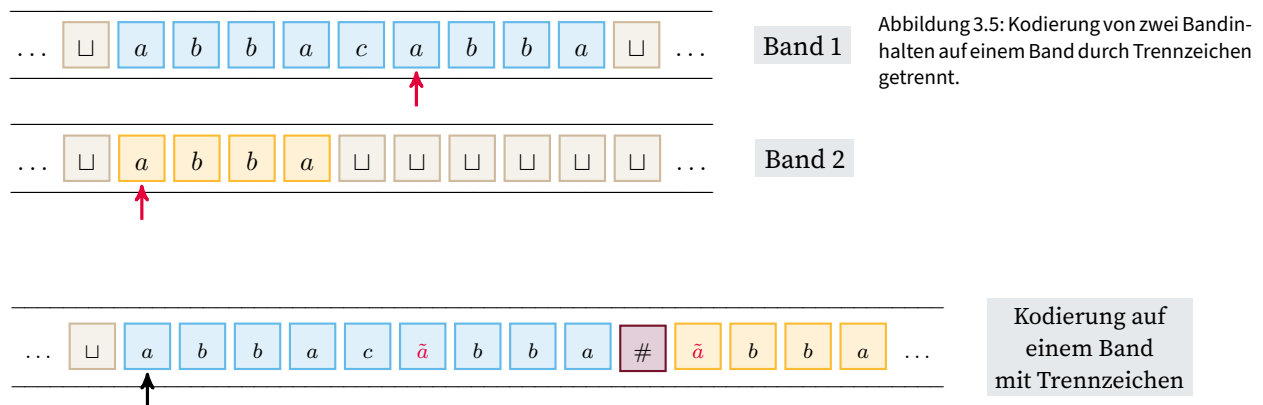
Eine erste Möglichkeit ist wie in Abbildung 3.4 dargestellt den Inhalt zweier untereinander liegender Zellen in einem Tupel zu kodieren. Zusätzlich muss noch gespeichert werden, ob sich auf der jeweiligen Zelle der Kopf des entsprechenden Bandes befindet oder nicht.

Die simulierende Turing-Maschine befindet sich links vom Bandinhalt, läuft nach rechts, bis alle Kopfmarker gefunden wurden, bestimmt den Nachfolgezustand, Kopfbewegungen und neuen Bandinhalten und läuft zurück nach ganz links und führt währenddessen die Updates aus.

In Abbildung 3.5 ist eine Alternative dargestellt, bei der einfach beide Bandinhalte durch ein Trennzeichen getrennt auf einem Band stehen. Da das Band unendlich lang ist, ist genug Platz. Die Position



der Köpfe muss trotzdem noch in der Zelle mit markiert werden. Eine Besonderheit hierbei ist, dass der Bereich des ersten Bandes nach links kopiert werden muss, wenn der Kopf des ersten Bandes das Trennzeichen erreicht und entsprechend der Bereich des zweiten Bandes nach rechts kopiert werden muss, wenn der 2. Kopf das Trennzeichen erreicht.



Nichtdeterministische Turing-Maschinen

Wie bei den endlichen Automaten und Kellerautomaten kann man auch bei Turing-Maschinen Nichtdeterminismus erlauben. Solche Maschinen sind zwar nicht direkt praktisch durch Computer zu implementieren, aber sie erlauben häufig eine kurze und einfache Beschreibung eines algorithmischen Problems. Es ist häufig einfacher eine gegebene Lösung zu überprüfen, als die Lösung selbst zu finden. Es ist schwierig eine große Zahl in Faktoren zu zerlegen aber einfach zu prüfen, ob eine gegebene Zerlegung korrekt ist.

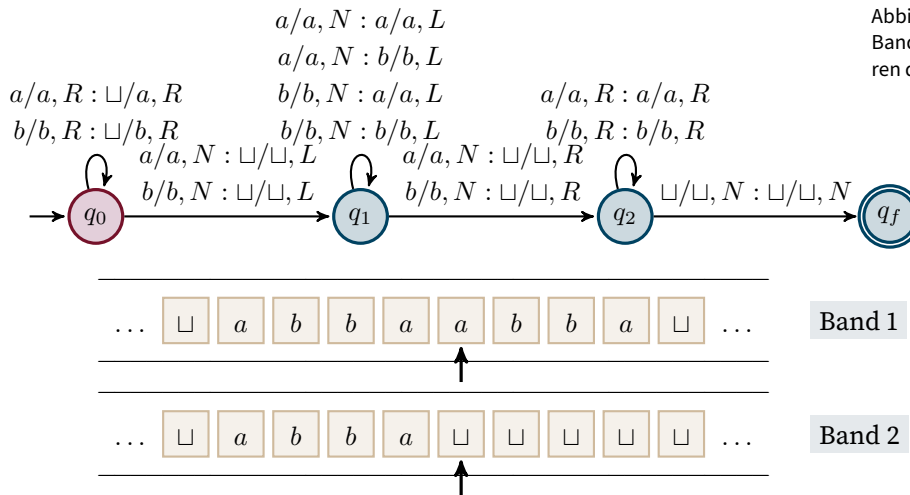
Nichtdeterministische Turing-Maschinen raten eine Lösung und prüfen dann, ob die Lösung korrekt ist und akzeptieren nur in diesem Fall. Die Turing-Maschine in Abbildung 3.6 akzeptiert die Sprache

Nichtdeterministische Turing-Maschinen (Non Deterministic Turing Machines)

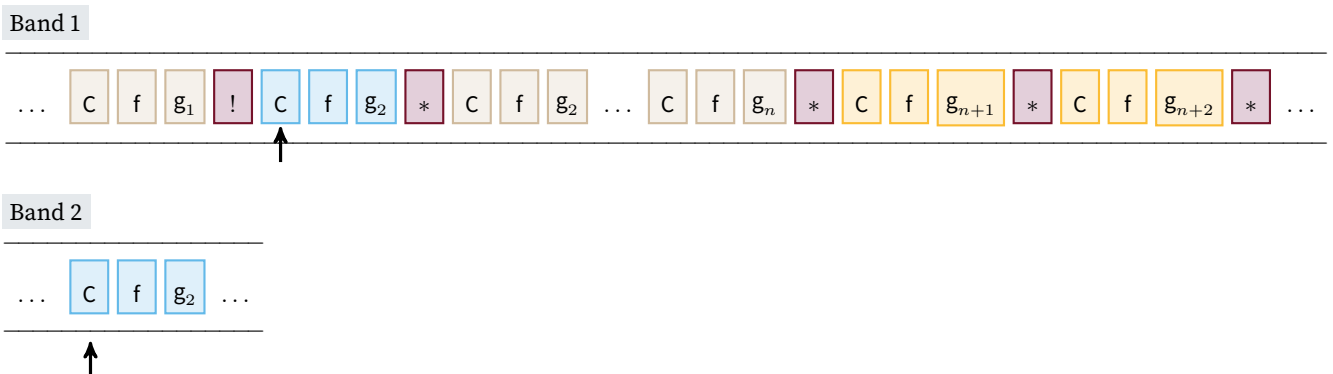


<https://youtu.be/6qEAd56IULY>
(<https://youtu.be/ZsXvtB0VAPw>)

$\{ww \mid w \in \{a,b\}^*\}$. Im Gegensatz zu der in Abbildung 3.3 akzeptierten Sprache besteht die Schwierigkeit darin, dass wir das c nicht als Marker nutzen können. Eine nichtdeterministische Turing-Maschine kann die Mitte einfach raten und dann prüfen, ob beide Teile gleich sind. Die Turing-Maschine arbeitet nach dem Guess-And-Check-Ansatz bei dem zuerst eine Lösung geraten und danach geprüft wird. Wichtig ist, dass auf den Prüf-Schritt nicht verzichtet werden kann.



Nichtdeterministische Turing-Maschinen können durch deterministische Turing-Maschinen simuliert werden. Dazu geht die deterministische Turing-Maschine einfach alle Konfigurationen in Breitensuche durch. Tiefensuche ist nicht möglich, weil die simulierte nichtdeterministische Turing-Maschine einen Pfad haben kann, der nicht terminiert und einen anderen Pfad, der akzeptiert. Würde der Simulator in Tiefensuche den unendlichen Pfad verfolgen, würde er den akzeptierten Pfad nicht finden. Für eine Breitensuche verwaltet



die simulierende Turing-Maschine wie in Abbildung 3.7 auf dem ersten Band eine Queue, in der alle noch zu bearbeitenden Konfigurationen durch ein Trennzeichen wie $*$ getrennt aufgelistet sind und deren Kopf durch ein anderes Sonderzeichen wie $!$ markiert ist. Die simulierende Turing-Maschine kopiert immer jeweils die nächste Konfiguration (Cf_{g_2}) aus der Queue auf das Arbeitsband, bestimmt alle mög-

lichen Nachfolgekonfigurationen (Cfg_{n+1} und Cfg_{n+2}) und hängt sie hinten an die Queue an.

3.2 Unentscheidbare Probleme

In der Softwareentwicklung ist man im Allgemeinen daran interessiert, dass die entwickelte Software auch tut was sie soll. Wenn man eine Funktion aufruft, möchte man meist, dass die Funktion irgendwann endet und der Kontrollfluss zum Aufrufer zurückkehrt. Da wir Turing-Maschinen als formales Modell für Funktionen in einem Programm oder für das ganze Programm ansehen, entspricht die Frage ob eine Funktion irgendwann terminiert der Frage, ob eine Turing-Maschine hält. Dieses Problem ist im Allgemeinen unentscheidbar. Das bedeutet, dass wir zwar für einzelne spezielle Funktionen prüfen und auch beweisen können, dass sie terminieren, aber wir können kein Programm schreiben, dass uns diese Fragestellung immer korrekt beantwortet. Der Beweis beruht darauf, dass man Turing-Maschinen auf ihren eigenen Code ansetzt. Dieser Trick ist auf den ersten Blick überraschend aber in der Praxis ist das gar nicht so ungewöhnlich. Compiler, IDEs und Tools zur Analyse von Programmen arbeiten mit Programmen als Input. Es spricht auch beispielsweise nichts dagegen mit der IDE Eclipse den Code von Eclipse zu bearbeiten.

Codierung von Turing-Maschinen

Turing-Maschinen können genau wie Programme codiert werden. Eine Turing-Maschine ist ein Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$. Diese Tupel kann man hinschreiben und die Zeichenkette binär codieren - wie bei Programmen auch. So erhält man zu jeder Turing-Maschine eine Codierung. Wenn man möchte, dass die Codierung bijektiv ist, kann man weiter annehmen, dass alle Binärstring, die keine syntaktisch korrekte Turing-Maschine kodieren, eine feste Turing-Maschine codieren sollen.

Definition 83: Codierung einer Turingmaschine

Mit M_w bezeichnen wir die Turing-Maschine zur Binärcodierung $w \in \{0, 1\}^*$.

Außerdem können Turing-Maschinen andere Turing-Maschinen simulieren, ganz analog wie eine virtuelle Maschine einen Rechner simuliert oder ein Interpreter ein Programm ausführt.

Das Halteproblem

Eine Turing-Maschine hält, wenn sie nicht weitermachen kann.

Definition 84: Halten von Turing-Maschinen

Eine Turing-Maschine **hält**, wenn Sie für aktuellen Zustand und aktuelles Bandzeichen keine Transition hat. Wir nehmen an, dass dies für alle akzeptierenden Zustände gilt.

Turing-Maschinen - Halten (Turing Machines - Halting)



<https://youtu.be/C7GHANuwkww>
(<https://youtu.be/LVx0KKpuhoc>)

Das Halten kann auch als Akzeptanzkriterium anstelle von akzeptierenden Zuständen verwendet werden.

Definition 85: Akzeptanz durch Halten

Eine Turing-Maschine M **akzeptiert** ein Wort w **durch Halten**, wenn die Turing-Maschine angesetzt auf das Wort irgendwann hält. Wir schreiben das kurz als $M \downarrow w$.

Akzeptanz durch Endzustand und Akzeptanz durch Halten sind äquivalente Ansätze.

Aus einer Turing-Maschine, die durch Halten akzeptiert, kann eine Turing-Maschine konstruiert werden, die durch Endzustand akzeptiert, indem einfach alle undefinierten Übergänge in einen neuen Endzustand geleitet werden.

Umgekehrt kann aus einer Turing-Maschine, die durch Endzustand akzeptiert eine Turing-Maschine konstruiert werden, die durch Halten akzeptiert, in dem alle undefinierten Übergänge von Nicht-Endzuständen in einen neuen Nicht-Endzustand mit Endlosschleife geleitet werden und alle ausgehenden Kanten in Endzuständen entfernt werden (sofern vorhanden).

Für Akzeptanz muss die Turing-Maschine nur halten, wenn das Wort in der Sprache ist. Falls das Wort nicht in der Sprache ist, weiß der Beobachter nicht, ob er länger warten muss oder nicht. Für Entscheidbarkeit möchte man, dass man sowohl im positiven als auch im negativen Fall eine Antwort erhält.

Definition 86: Entscheidbarkeit

Eine Sprache L (bzw. ein Problem) heißt **entscheidbar** (engl. *decidable*) genau dann wenn es eine Turing-Maschine M gibt, die angesetzt auf ein Wort w als Ergebnis 1 liefert wenn $w \in L$ gilt und sonst 0 liefert.

Definition 87: Spezielle Halteproblem

Das **spezielle Halteproblem** ist das Problem, ob eine Turing-Maschine angesetzt auf Ihre **eigene** Kodierung hält.

Formal: Das spezielle Halteproblem ist die Sprache

$$K = \{w \in \{0, 1\}^* \mid M_w \text{ angesetzt auf } w \text{ hält}\}$$

Theorem 88: Unentscheidbarkeit des speziellen Halteproblems

Das spezielle Halteproblem ist unentscheidbar.

Beweis 89: Unentscheidbarkeit des speziellen Halteproblems

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die das Problem K entscheidet. Für diese Turing-Maschine M angesetzt

Akzeptanz durch Halten und Endzustand (Accepting by Halting or Final State)



<https://youtu.be/ED2dPBTP8MI>
(<https://youtu.be/q0hl7nc7fy0>)

Entscheidbarkeit (Decidability)



https://youtu.be/PKz_FGxzD7k
(<https://youtu.be/AVJKfXA01TA>)

Das Spezielle Halteproblem ist unentscheidbar (Undecidability of a halting problem)



https://youtu.be/mUonpuXji_s
(<https://youtu.be/pZbUFbcGjtg>)

auf Eingabe x gilt:

$$M \text{ angesetzt auf } x \text{ liefert } \begin{cases} 1 & \text{falls } M_x \text{ angesetzt auf } x \text{ hält} \\ 0 & \text{falls } M_x \text{ angesetzt auf } x \text{ nicht hält} \end{cases} \quad (3.1)$$

Konstruiere aus M eine Turing Maschine M' , die immer dann nicht hält, wenn die Eingabeturingmaschine auf ihren eigenen Code halten würde. M' tut dazu folgendes:

1. Simuliere M (Ergebnis steht auf Band)
2. Prüfe Ergebnis von M
 - (a) Falls Band = 1 $\Rightarrow$ starte Endlosschleife
 - (b) Falls Band = 0 $\Rightarrow$ stoppe

Das Problem ergibt sich, wenn man M' auf seinen eigenen Code ansetzt. Sei nun w' der Code der Turing-Maschine M' . Wir betrachten, was passiert, wenn M' auf w' angesetzt wird.

Nach Konstruktion von M' (2 (a) und 2 (b)) gilt:

$$M' \text{ angesetzt auf } w' \text{ hält} \iff M \text{ angesetzt auf } w' \text{ liefert } 0$$

Aufgrund der Eigenschaften von M nach (3.1) gilt für $x = w'$:

$$\iff M_{w'} \text{ angesetzt auf } w' \text{ hält nicht}$$

Da w' die Codierung von M' ist, gilt $M' = M_{w'}$ und damit auch:

$$\iff M' \text{ angesetzt auf } w' \text{ hält nicht}$$

Dieser Widerspruch kann nur daran liegen, dass die Annahme zur Existenz von M falsch war. Also ist das spezielle Halteproblem unentscheidbar.

Weitere unentscheidbare Probleme

Definition 90: Leerbandhalteproblem

Das **Leerbandhalteproblem** ist das Problem ob eine Turing-Maschine angesetzt auf das leere Band hält.

Formal: Das Leerbandhalteproblem ist die Sprache

$$H_0 = \{w \in \{0, 1\}^* \mid M_w \text{ angesetzt auf das leere Band hält}\}$$

Theorem 91: Unentscheidbarkeit des Leerbandhalteproblems

Das Leerbandhalteproblem ist unentscheidbar.

Reduktion - Kulinarische Einführung (Reduction - A Culinary Introduction)



<https://youtu.be/814iptNr7oc>
(<https://youtu.be/SjE0yBQUrwY>)

Reduktion - Einführung (Reduction - Introduction)



https://youtu.be/0xf_jk893vk
(<https://youtu.be/hopVksj4-b0>)

Reduktion - Formale Definition (Reduction - Formal Definition)



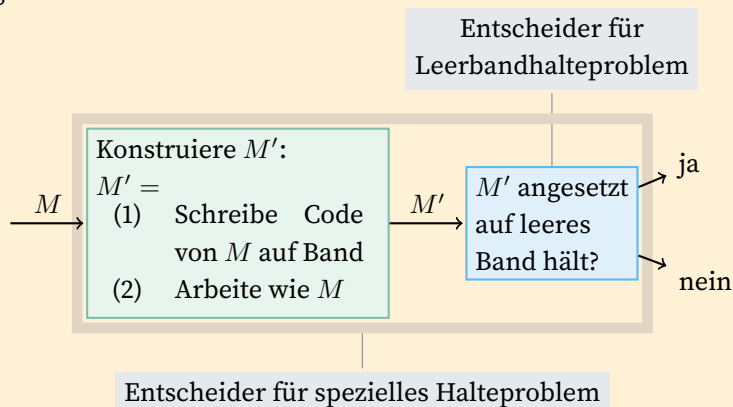
<https://youtu.be/UufdnF5fXvI>
(<https://youtu.be/ks3Jex0ht3A>)

Beweis 92: Unentscheidbarkeit des Leerbandhalteproblems

Wir zeigen: Wenn wir das Leerbandhalteproblem entscheiden können, dann können wir auch das spezielle Halteproblem entscheiden.

Da wir schon wissen, dass das spezielle Halteproblem unentscheidbar ist, haben wir dadurch gezeigt, dass das Leerbandhalteproblem ebenfalls unentscheidbar ist.

Wir nehmen an, dass wir einen Entscheider für das Leerbandhalteproblem haben. Diesen Entscheider wollen wir nutzen, um das spezielle Halteproblem zu entscheiden. Dazu müssen wir eine Instanz M des speziellen Halteproblems – also eine Turing-Maschine von der wir wissen wollen, ob sie auf ihren eigenen Code hält – in eine Instanz M' des Leerbandhalteproblems umwandeln, so dass M angesetzt auf den eigenen Code genau dann hält, wenn M' angesetzt auf das leere Band hält.



Wie in dem Schema dargestellt erzeugen wir aus der Turing-Maschine M eine neue Turing-Maschine M' , die zuerst den Code von M auf das Band schreibt und danach M startet.

Diese neue Turing-Maschine M' verhält sich angesetzt auf das leere Band – d.h. mit leerem Band als Eingabe – genauso wie die Turing-Maschine M angesetzt auf den Code von M . Also hält M' angesetzt auf das leere Band genau dann wenn M angesetzt auf den eigenen Code hält. Wir haben das spezielle Halteproblem auf das Leerbandhalteproblem *reduziert*.

Wenn es nun einen Entscheider für das Leerbandhalteproblem gäbe, könnten wir für eine Turing-Maschine M entscheiden, ob sie angesetzt auf ihren eigenen Code hält, indem wir zuerst M' konstruieren und dann M' in den Entscheider für das Leerbandhalteproblem stecken. Da das spezielle Halteproblem unentscheidbar ist, muss auch das Leerbandhalteproblem unentscheidbar sein.

Anstatt der schematischen Darstellung im Beweis, kann man das Vorgehen auch mit Programmcode verdeutlichen. Anstelle eines Eingabebandes nutzen wir eine Variable `band`. Wir nehmen an, es gäbe eine Funktion `leerband_hp(code_l)`, die `True` liefert, wenn der Programmcode in Parameter `code_l` für `band = ''` hält und `False` falls er nicht hält. Unter dieser Annahme können wir eine

Leerbandhalteproblem (*Halting on the Empty Tape*)



<https://youtu.be/ttbMJtNsZEQ>
(https://youtu.be/AG_CinFL5kM)

Funktion `spezielles_hp(code_s)` schreiben, die prüft, ob `code_s` hält, wenn die Variable `band` den Wert `code_s` hat. Dazu schreibt sie einfach die Zuweisung `'band = '' + code_s + '\\n'` als erste Anweisung vor den restlichen Code.

Reduktion des speziellen Halteproblems auf das Leerbandhalteproblem

```
def leerband_hp(code_l):
    # Entscheidet das Leerbandhalteproblem
    # also ob code_l haelt, wenn die Variable
    # band den Wert '' hat.
    band = ''
    # magic happens...
    return True # or False

def spezielles_hp(code_s):
    # Entscheidet das spezielle Halteproblem
    # fuer code_s, d.h. ob der Programmcode
    # code_s haelt, wenn die Variable band den
    # Wert code_s hat.
    # Dazu wird die Funktion leerband_hp genutzt.
    code_l = 'band = '' + code_s + '\\n' + code_s
    return leerband_hp(code_l)
```

Dieser `code_l` verhält sich beim Aufruf jetzt unabhängig vom Inhalt von `band` immer so wie `code_s` falls `band` den Wert `code_s` hat. Wenn wir also wissen, ob `code_l` für das leere `band` hält, wissen wir auch, ob `code_s` hält, falls `band` den Wert `code_s` hat. Letzteres ist aber schon als unentscheidbar bekannt.

Eine Reduktion kann formal durch eine Funktion beschrieben werden, die eine Instanz des einen Problems auf eine Instanz des anderen Problems abbildet.

Definition 93: Reduktion – formal

Seien $A \subseteq \Sigma_1^*$ und $B \subseteq \Sigma_2^*$ Sprachen. Dann heißt A auf B *reduzierbar* $A \leq B$ falls es eine totale und berechenbare Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, so dass für alle $x \in \Sigma_1^*$ gilt:

$$x \in A \iff f(x) \in B$$

Wichtig an dieser Definition sind drei Punkte:

1. Die Funktion f muss berechenbar sein. Andernfalls könnte die Funktion f bereits ein unberechenbares Problem lösen.
2. Die Funktion muss total sein, damit für alle Instanzen des Problems A auch ein Bild existiert.
3. Es muss eine Äquivalenz zwischen den Instanzen der Probleme A und B bestehen und nicht nur eine Implikation. Das ist notwendig, damit beide Fälle $x \in A$ und $x \notin A$ abgedeckt sind.

Wir werden meistens keine Funktion angeben, sondern eine Konstruktion, wie eine Instanz des bekannten Problems auf das neue Pro-

blem reduziert werden können.

Theorem 94: Reduktion

1. Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.
2. Wenn A unentscheidbar ist und $A \leq B$, dann ist auch B unentscheidbar.

Beweis 95: Reduktion

Wir wollen $x \in A$ entscheiden. Wegen $A \leq B$ gibt es eine berechenbare Funktion f mit $x \in A \iff f(x) \in B$. Wir berechnen $f(x)$. Wir entscheiden $f(x) \in B$. Das ist möglich, weil B entscheidbar ist. Nun wissen wir wegen $x \in A \iff f(x) \in B$ auch die Antwort auf die Frage ob $x \in A$ gilt. Der zweite Teil ist nur eine logische Umformung (Kontraposition) des ersten Teils.

Definition 96: Allgemeine Halteproblem

Das allgemeine Halteproblem ist das Problem, ob eine Turing-Maschine M angesetzt auf ein Wort x hält.

Formal: Das Halteproblem ist die Sprache

$$H = \{w\#x \mid M_w \text{ angesetzt auf } x \text{ hält}\}$$

Theorem 97: Unentscheidbarkeit des allgemeinen Halteproblems

Das allgemeine Halteproblem ist unentscheidbar.

Beweis 98: Unentscheidbarkeit des Leerbandhalteproblems

Die Funktion $f : \{0, 1\}^* \rightarrow \{0, 1, \#\}^*$ mit $f(w) = w\#w$ ist eine Reduktion. Sie reduziert das spezielle Halteproblem auf das allgemeine Halteproblem. Die Funktion ist offensichtlich total und berechenbar.

Außerdem gilt

$$\begin{aligned} w \in K &\iff M_w \text{ angesetzt auf } w \text{ hält} \\ &\iff w\#w \in H \end{aligned}$$

Definition 99: Leerheitsproblem

Das Leerheitsproblem ist das Problem ob eine Turing-Maschine für keine Eingabe hält.

Das Leerheitsproblem ist die Sprache

$$E = \{w \mid M_w \text{ hält für keine Eingabe}\}$$

Theorem 100: Unentscheidbarkeit des Leerheitsproblems

Das Leerheitsproblem ist unentscheidbar.

Das allgemeine Halteproblem
(Undecidability of the general halting problem)



https://youtu.be/Msws_VAPm5A
(<https://youtu.be/yydMHJa8iyw>)

Leerheitsproblem (The Emptiness Problem)



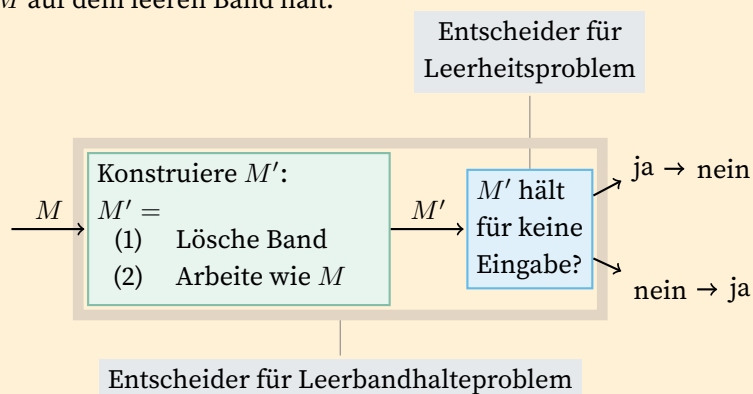
<https://youtu.be/PZD6yNvkd94>
(<https://youtu.be/-p4uHVpjcxU>)

Beweis 101: Unentscheidbarkeit des Leerheitsproblems

Wir zeigen: Wenn wir das Leerheitsproblem entscheiden können, dann können wir auch das Leerbandhalteproblem entscheiden.

Da wir schon wissen, dass das Leerbandhalteproblem unentscheidbar ist, haben wir dadurch gezeigt, dass das Leerheitsproblem ebenfalls unentscheidbar ist.

Wir nehmen an, dass wir einen Entscheider für das Leerheitsproblem haben. Diesen Entscheider wollen wir nutzen, um das Leerbandhalteproblem zu entscheiden. Dazu müssen wir eine Instanz M des Leerbandhalteproblem – also eine Turing-Maschine von der wir wissen wollen, ob sie auf dem leeren Band hält - in eine Instanz M' des Leerheitsproblem umwandeln. Aus der Antwort auf die Frage, ob M' für keine Eingabe hält müssen wir ableiten können, ob M auf dem leeren Band hält.



Wie in dem Schema dargestellt erzeugen wir aus der Turing-Maschine M eine neue Turing-Maschine M' , die zuerst das Band löscht und danach wie M arbeitet.

Diese neue Turing-Maschine M' verhält sich für alle Eingaben wie M mit dem leeren Band als Eingabe. Wenn M' also für keine Eingabe hält, dann hält M nicht auf das leere Band und wenn M' für eine Eingabe hält, dann hält M auf dem leeren Band. Wir sehen, dass wir das Ergebnis des Entscheiders für das Leerheitsproblem noch einmal negieren müssen.

Die Konstruktion von M' aus M ist berechenbar und damit haben wir das Leerbandhalteproblem auf das Leerheitsproblem reduziert und so gezeigt, dass das Leerheitsproblem unentscheidbar ist.

Auch dieses Reduktion kann man leicht in Python darstellen. Wenn es eine Funktion `leerheit` gibt, dann kann man die leicht nutzen, um eine Funktion `leerband_hp` zu schreiben.

Reduktion des speziellen Halteproblems auf das Leerbandhalteproblem

```

def leerheit(code_lh):
    # Entscheidet das Leerheitsproblem
    # fuer code_lh, d.h. ob der Programmcode
    # code_lh für keinen Wert der Variable band haelt.
    # magic happens
  
```

```
def leerband_hp(code_l):
    # Entscheidet das Leerbandhalteproblem
    # also ob code_l haeft, wenn die Variable
    # band den Wert '' hat.
    # Dazu wird die Funktion leerheit genutzt.
    code_lh = 'band = ""\n' + code_l
    return leerheit(code_lh)
```

Definition 102: Äquivalenzproblem

Das Äquivalenzproblem ist das Problem ob zwei Turing-Maschine die gleiche Sprache akzeptieren.

Das Äquivalenzproblem ist die Sprache

$$EQ = \{w_1 \# w_2 \mid M_{w_1} \text{ hält für } w \text{ gdw. } M_{w_2} \text{ hält für } w\}$$

Theorem 103: Unentscheidbarkeit des Äquivalenzproblems

Das Äquivalenzproblem ist unentscheidbar.

Beweis 104: Unentscheidbarkeit des Äquivalenzproblems

Um zu zeigen, dass das Äquivalenzproblem unentscheidbar ist, brauchen wir ein bekanntes unentscheidbares Problem, das wir darauf reduzieren können. Wir probieren es mit dem Leerheitsproblem.

Wir starten mit einer Instanz des Leerbandhalteproblems, d.h. einer Turing-Maschine M (bzw. einer Codierung davon) von der wir wissen wollen, ob sie für keine Eingabe hält.

Wir müssen daraus eine Instanz des Äquivalenzproblems machen, d.h. zwei Turing-Maschinen M' und M'' finden, so dass die Äquivalenz aus Definition 93 gilt. Es ist hilfreich sich diese Äquivalenz hinzuschreiben.

$$\begin{aligned} &M \text{ hält für keine Eingabe} \\ \iff &M' \text{ hält für die gleichen Wörter } w \text{ wie } M'' \end{aligned}$$

Wenn man diese Äquivalenz genauer ansieht überlegt, wie man von der Formulierung auf der linken Seite zur rechten Seite kommen kann, fällt folgendes auf: M hält für keine Eingabe genau dann, wenn M die gleichen Wörter akzeptiert wie eine Turing-Maschine, die für keine Eingabe hält.

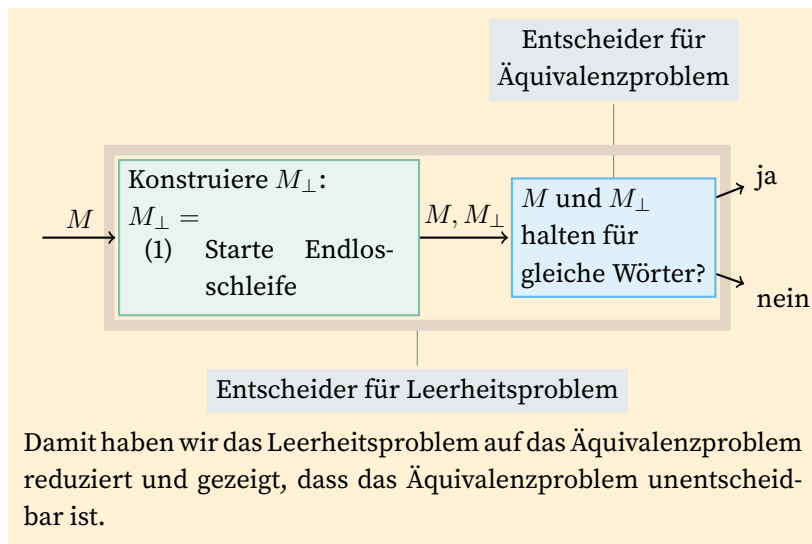
Es ist zwar unentscheidbar, ob eine Turing-Maschine für keine Eingabe hält, aber wir können leicht eine Turing-Maschine $M_\perp$ konstruieren, die diese Eigenschaft hat. Für Programmcode kann man im allgemeinen nicht entscheiden, ob er für keine Eingabe hält. Aber Sie können leicht Code mit dieser Eigenschaft herstellen, indem Sie einfach eine Endschlosschleife schreiben.

Für die Reduktion können wir also $M' = M$ und $M'' = M_\perp$ setzen.

Äquivalenzproblem (Testing Equivalence)



<https://youtu.be/eLURKNbQrrI>
(<https://youtu.be/3TvKv9QSDEE>)



Der Satz von Rice

Im vorherigen Abschnitt haben gesehen, dass viele Fragen zum Ein-/Ausgabeverhalten von Turing-Maschinen unentscheidbar sind. Es sind nicht nur viele Eigenschaften, sondern fast alle. Das ist die Aussage des Satzes von Rice. Wir stellen den Satz einmal in der Funktionsversion und einmal in der Sprachversion vor.

Theorem 105: Satz von Rice (Funktionsversion)

Sei $\mathcal{R}$ die Klasse aller Turing-berechenbaren Funktionen. Sei S eine nichttriviale Teilmenge, d.h. $S \neq \emptyset$ und $S \neq \mathcal{R}$. Dann ist unentscheidbar, ob die von einer TM berechnete Funktion in S liegt.

Formal:

$$C(S) = \{w \mid M_w \text{ berechnet Funktion in } S\}$$

ist unentscheidbar.

Da Turing-Maschinen äquivalent zu Programmen in Standard-Programmiersprachen sind, hat der Satz weitreichende Konsequenzen. Er bedeutet, dass egal wie simpel die Eigenschaft ist, die wir prüfen möchten, ist es im Allgemeinen unentscheidbar, ob ein Programm diese Eigenschaft hat. Die einzigen zwei Ausnahmen sind die Eigenschaften "Das Programm tut irgendwas" und "Das Programm tut nichts". Da Programme immer irgendwas tun – auch wenn sie nie halten –, ist die Antwort im ersten Fall immer wahr und im zweiten immer falsch.

Beweisidee: Für den Beweis reduzieren wir das Leerbandhalteproblem auf das Problem im Satz von Rice. Die Turing-Maschine M für die wir das Leerbandhalteproblem entscheiden möchten, lassen wir einfach auf dem leeren Band loslaufen. Falls diese Turing-Maschine hält, starten wir eine andere Turing-Maschine Q , die eine feste Funktion berechnet. Falls M auf dem leeren Band nicht hält, wird Q nie gestartet und die Reihenschaltung hält ebenfalls nicht. In diesem Fall berechnet die Reihenschaltung die überall undefinierte Funktion $\perp$.

Der Satz von Rice (Rice's Theorem)



<https://youtu.be/XPQB1Tei60Y>
(https://youtu.be/_XHqKkQx8)

Falls M auf dem leeren Band hält, berechnet die Turing-Maschine Q . Nun müssen wir Q lediglich so wählen, dass entweder Q in S liegt und $\perp$ nicht oder umgekehrt.

Beweis 106: Satz von Rice

Sei S eine beliebige nichttriviale Teilmenge der Turing-berechenbaren Funktionen. Sei $\perp$ die überall undefinierte Funktion. Diese Funktion wird z.B. von der Turing-Maschine berechnet, die direkt eine Endlosschleife beginnt.

Für die Reduktion benötigen wir eine Instanz des Leerbandhalteproblems. Sei also M eine Turing-Maschine, für die wir das Leerbandhalteproblem entscheiden wollen.

1. Fall $\perp \notin S$ Da die Menge S nicht leer ist, muss mindestens eine Funktion q in S liegen. Da alle Funktionen in S Turing-berechenbar sind, muss es auch für q mindestens eine Turing-Maschine Q geben, die q berechnet.

Wir konstruieren eine Turing-Maschine M' , die folgendes tut:

1. Sichere die Eingabe x auf ein zweites Band und lösche das erste Band.
2. Simuliere M auf dem leeren ersten Band.
3. Falls M irgendwann hält, lösche das erste Band, schreibe x zurück auf das erste Band.
4. Simuliere Q auf dem ersten Band.

Diese Turing-Maschine M' hat folgendes Verhalten.

- Falls M auf dem leeren Band nicht hält, hält auch M' unabhängig von der Eingabe nicht. In diesem Fall berechnet M die überall undefinierte Funktion $\perp$. Da $\perp \notin S$, berechnet M' also eine Funktion die nicht in S liegt.
- Falls M auf dem leeren Band hält, verhält sich M' wie Q , d.h. berechnet die Funktion q , die in S liegt.

Damit hat M' die Eigenschaft, dass sie eine Funktion in S berechnet, genau dann wenn M auf dem leeren Band nicht hält. Wenn wir also das Problem aus dem Satz von Rice entscheiden können, können wir auch das Leerbandhalteproblem entscheiden.

Diese Reduktion ist rechts auch in Python-Code veranschaulicht. Für die Veranschaulichung nehme ich an, dass q die Funktion ist, die $n \mapsto n + 42$ berechnet.

2. Fall $\perp \in S$ Dieser Fall ist weitgehend analog. Wir müssen nur q diesmal anders wählen. Da die Menge S die Menge aller Turing-berechenbaren Funktionen ist, muss es mindestens eine Turing-berechenbare Funktion q geben, die **nicht** in S liegt. Da q Turing-berechenbar ist, muss es für sie mindestens eine Turing-Maschine Q geben, die sie berechnet.

Satz von Rice - Fall 1

```
def rice(code_r):
    # Entscheidet, ob code_r
    # eine Funktion in S
    # berechnet
    # ...
    return True # or False

def leerband_hp(code):
    # Entscheidet das
    # Leerbandhalteproblem
    # mit Hilfe der Funktion
    # rice.
    #
    # Code fuer feste
    # Funktion q
    qcode = \
        'band = band + 42'
    code_r = \
        'band1 = band\n' \
        'band = ""\n' \
        'exec("'" + \
        code + '"')\n' \
        'band = band1\n' \
        'exec("'" + \
        qcode + '"')\n'
    return not rice(code_r)
```

Wir konstruieren genau wie für Fall 1 die Turing-Maschine M' . Diese Turing-Maschine M' hat jetzt folgendes Verhalten.

- Falls M auf dem leeren Band nicht hält, hält auch M' unabhängig von der Eingabe nicht. In diesem Fall berechnet M die überall undefinierte Funktion $\perp$. Da $\perp \in S$, berechnet M' also eine Funktion die in S liegt.
- Falls M auf dem leeren Band hält, verhält sich M' wie Q , d.h. berechnet die Funktion q , die nicht in S liegt.

Damit hat M' die Eigenschaft, dass sie eine Funktion in S berechnet, genau dann wenn M auf dem leeren Band hält. Wenn wir also das Problem aus dem Satz von Rice entscheiden können, können wir auch das Leerbandhalteproblem entscheiden.

Wir bringen den gleichen Satz noch in der Sprachversion. Da wir häufig die von Turing-Maschinen akzeptierten Sprachen statt der berechneten Funktionen betrachten.

Theorem 107: Satz von Rice (Sprachversion)

Sei S eine nichttriviale Menge von Turing-akzeptierbaren Sprachen, d.h. $S \neq \emptyset$ und S ist nicht die Menge aller Turing-akzeptierbaren Sprachen. Dann ist unentscheidbar, ob die von einer TM akzeptierte Sprache in S liegt.

Formal:

$$L(S) = \{w \mid M_w \text{ akzeptiert Sprache in } S\}$$

ist unentscheidbar.

Der Satz von Rice (Sprachversion) (Rice's Theorem (Language Version))



<https://youtu.be/DdgrcXuk7C0>
(<https://youtu.be/8RStUYyjlU>)

Beweis 108: Satz von Rice

Sei S eine beliebige nichttriviale Teilmenge der Turing-akzeptierbaren Sprachen. Anstelle der überall undefinierten Funktion nutzen wir die leere Sprache. Die leere Sprache $\emptyset$ wird z.B. von der Turing-Maschine akzeptiert, die direkt eine Endlosschleife beginnt.

Für die Reduktion benötigen wir eine Instanz des Leerbandhalteproblems. Sei also M eine Turing-Maschine, für die wir das Leerbandhalteproblem entscheiden wollen.

1. Fall $\emptyset \notin S$ Da die Menge S nicht leer ist, muss mindestens eine Sprache L in S liegen. Da alle Sprachen in S Turing-akzeptierbar sind, muss es auch für L mindestens eine Turing-Maschine M_K geben, die L akzeptiert.

Wir konstruieren eine Turing-Maschine M' , die folgendes tut:

1. Sichere die Eingabe x auf ein zweites Band und lösche das erste Band.
2. Simuliere M auf dem leeren ersten Band.
3. Falls M irgendwann hält, lösche das erste Band, schreibe x zurück auf das erste Band.

4. Simuliere M_L auf dem ersten Band.

Diese Turing-Maschine M' hat folgendes Verhalten.

- Falls M auf dem leeren Band nicht hält, hält auch M' unabhängig von der Eingabe nicht. In diesem Fall akzeptiert M die leere Sprache $\emptyset$. Da $\emptyset \notin S$, akzeptiert M' also eine Sprache die nicht in S liegt.
- Falls M auf dem leeren Band hält, verhält sich M' wie M_L , d.h. akzeptiert L , die in S liegt.

Damit hat M' die Eigenschaft, dass sie eine Sprache in S akzeptiert, genau dann wenn M auf dem leeren Band nicht hält. Wenn wir also das Problem aus dem Satz von Rice entscheiden können, können wir auch das Leerbandhalteproblem entscheiden.

2. Fall $\perp \in S$ Dieser Fall ist wieder völlig analog.

In der Sprachversion ist wichtig zwischen $\emptyset \in S$ und $\emptyset = S$ zu unterscheiden. Der erste Fall ist möglich, da S eine Menge von Sprachen ist und die leere Sprache eine Turing-akzeptierbare Sprache ist. In diesem ersten Fall ist insbesondere S nicht leer, weil ja zumindest die leere Sprache in der Menge ist.

Beispiel 109: Folgerungen aus dem Satz von Rice

Aus dem Satz von Rice folgen viele Resultate direkt, z.B.

- Es ist unentscheidbar, ob zwei Programme das gleiche tun.
- Es ist unentscheidbar, ob ein Programm seine Spezifikation erfüllt.
- Es ist unentscheidbar, ob eine Turing-Maschine eine reguläre Sprache akzeptiert.

Anwendungen des Satz von Rice (Application of Rice's Theorem)



<https://youtu.be/aELKxfYs1j8>
(<https://youtu.be/XFMTb1v11J0>)

3.3 Semi-Entscheidbare Probleme

In Definition 86 wird Entscheidbarkeit definiert. Das Leerbandhalteproblem ist unentscheidbar, aber wir können eine Turing-Maschine einfach mal auf das leere Band loslaufen lassen und gucken was passiert. Falls die Turing-Maschine hält, wissen wir sicher, dass sie für das leere Band hält. Falls Sie noch nicht gehalten hat, müssen wir entweder länger warten oder sie wird nicht halten. In dem negativen Fall können wir keine definitive Aussage treffen. Das Problem ist unentscheidbar aber semi-entscheidbar.

Definition 110: Semi-Entscheidbarkeit / Rekursive Aufzählbarkeit

Eine Sprache L (bzw. ein Problem) heißt **semi-entscheidbar** (semi-decidable) oder **rekursiv aufzählbar** (recursively enumerable) genau dann wenn es eine Turing-Maschine M gibt, die angesetzt auf ein Wort

Semi-Entscheidbarkeit (Semi-Decidability)



<https://youtu.be/nkU0snYeh0g>
(<https://youtu.be/P9mLVmMH5Gg>)

w als Ergebnis 1 liefert wenn $w \in L$ (für den Fall $w \notin L$ wird nichts vorausgesetzt).

Theorem 111: Entscheidbarkeit und Semi-Entscheidbarkeit

1. Wenn eine L entscheidbar ist, ist sie auch semi-entscheidbar.
2. Wenn L und $\bar{L}$ semi-entscheidbar sind, dann ist L entscheidbar.

Beweis 112: Entscheidbarkeit und Semi-Entscheidbarkeit

1. Diese Aussage folgt direkt aus der Definition.
2. Seien L und $\bar{L}$ semi-entscheidbar und M_1 und M_2 die Turing-Maschinen, die Sprachen semi-entscheiden. Wir konstruieren daraus eine Turing-Maschine M , die auf dem ersten Band M_1 und auf dem zweiten Band M_2 jeweils schrittweise abwechselnd simuliert, d.h. M_1 macht einen Schritt auf dem ersten Band, dann macht M_2 einen Schritt auf dem zweiten Band, usw.

Da die Eingabe entweder in L oder im Komplement $\bar{L}$ liegt, muss eine der Turing-Maschinen halten und eine 1 schreiben. Wenn es M_1 ist, dann liegt das Wort in L und der Simulator M kann eine 1 schreiben. Wenn es M_2 ist, dann liegt das Wort nicht in L und der Simulator M kann eine 0 schreiben,

3.4 Zeitkomplexität

Im vorherigen Abschnitt ging es um die Grenzen der Berechenbarkeit, also welche Probleme prinzipiell entscheidbar sind und welche nicht. Die Zeit, die ein Algorithmus benötigt hat, war dabei egal. Darum geht es in diesem Abschnitt. Unser Maschinenmodell ist noch die Turing-Maschine. Dann kann der Zeitverbrauch einfach definiert werden.

Definition 113: Zeitverbrauch

Der Zeitverbrauch einer Turing-Maschine ist die Anzahl der Schritte. Im nichtdeterministischen Fall, ist es die maximale Anzahl an Schritten im längsten Berechnungspfad – egal ob akzeptierender Pfad oder nicht –.

Die Anzahl der Schritte ist typischerweise von der Größe der Eingabe abhängig. Die Laufzeit wird deshalb typischerweise in der O -Notation angegeben.

Definition 114: O-Klasse

Sei $g : \mathbb{N} \rightarrow \mathbb{R}_0^+$ eine Funktion. Dann ist $O(g)$ die Klasse aller Funktionen f für die es Konstanten c und n_0 gibt, so dass

$$f(n) < c \cdot g(n) \text{ für alle } n > n_0$$

Diese Notation hat den Vorteil, dass sie eine obere Schranke für den Zeitverbrauch beschreibt und dabei auch von der konkreten Hard-

Komplexitätstheorie - Einstieg (Complexity – Introduction)



https://youtu.be/r6m_Nq3USrc
(<https://youtu.be/h6Bm6Aiw6A8>)

ware abstrahiert. Durch den konstanten Faktor in der Definition spielt es keine Rolle, ob eine Maschine doppelt so viele Operation pro Zeiteinheit machen kann oder wie genau die atomaren Operationen aussehen.

Die Probleme, deren Laufzeit durch ein Polynom beschränkt ist, bildet eine erste wichtige Klasse. Polynome sind Funktionen wie n , n^2 , n^3 oder allgemein $\sum_{i=0}^n a_i x^n$.

Definition 115: Die Klasse P

Die Klasse P ist die Klasse aller Sprachen L (Probleme), die in Polynomialzeit durch eine deterministische 1-Band-Turing-Maschine entschieden werden können.

Die Klasse P ist robust in Bezug auf das Maschinenmodell. Wir betrachten hier Turing-Maschinen, die für den Zugriff auf eine Speicherstelle $O(n)$ Schritte machen müssen. Würden wir stattdessen ein Modell betrachten, bei dem der Speicherzugriff in konstanter Zeit ($O(1)$) möglich ist, würden wir trotzdem die gleiche Klasse P erhalten. Wenn ein Algorithmus bei $O(1)$ -Speicherzugriff nur polynomiell viele Schritte machen muss, muss die Turing-Maschine maximal in jedem Schritt das Band entlang laufen. Die Laufzeit wäre immer noch ein Polynom.

Erlauben wir auch nichtdeterministische Turing-Maschinen, erhalten wir die Klasse NP .

Definition 116: Die Klasse NP

Die Klasse NP ist die Klasse aller Sprachen L (Probleme), die in Polynomialzeit durch eine nichtdeterministische 1-Band-Turing-Maschine entschieden werden können.

Wie schon bei den endlichen Automaten hatten wir gesehen, dass nichtdeterministische Maschinen sehr einfache Beschreibungen eines Problems erlauben. Der Nachteil ist, dass sie nicht direkt in Hardware implementiert werden können. Eine Möglichkeit ist natürlich alle Konfigurationen der nichtdeterministischen Maschinen durchzugehen. Dieser Ansatz hat aber exponentiellen Aufwand.

Die Beschreibung eines Problems mit Hilfe von Nichtdeterminismus ist deshalb häufig kürzer, weil es häufig einfacher ist, eine gegebene Lösung zu prüfen als sie zu finden. Eine nichtdeterministische Maschine würde eine Lösung raten und dann prüfen. Eine andere Charakterisierung der Klasse NP ist deshalb: NP ist die Klasse der Probleme deren Lösungen deterministisch in Polynomialzeit zu prüfen ist.

In der Berechenbarkeitstheorie hatten wir den Begriff der Reduktion benutzt, um Probleme zu vergleichen. Wenn ein Problem auf ein entscheidbares Problem reduzierbar ist, dann muss dieses Problem auch entscheidbar sein.

Wir nutzen die gleiche Idee jetzt, um die Komplexität zu vergleichen. Bei der polynomiellen Reduzierbarkeit soll die Reduktion höchstens polynomiellen Aufwand haben.

Die Klassen P und NP (Time Complexity – P and NP)



<https://youtu.be/VFqakTUBMX0>
(<https://youtu.be/k6eIJkA2ohw>)

Definition 117: Polynomielle Reduzierbarkeit

Seien A und B Sprachen. Dann heißt A auf B **polynomial reduzierbar** $A \leq_P B$ (polynomial time reducible) falls es eine totale und mit polynomialer Komplexität berechenbare Funktion f gibt mit

$$x \in A \iff f(x) \in B.$$

(Idee: A ist höchstens so schwierig wie B)

Wenn die dann ein Problem polynomiell auf ein Problem reduzieren können, das polynomiellen Aufwand hat, dann haben wir einen polynomiellen Algorithmus dieses Problem.

Theorem 118: Polynomielle Reduzierbarkeit

Sei B in polynomieller Zeit entscheidbar und sei die Sprache A auf B polynomiell reduzierbar. Dann ist A in polynomieller Zeit entscheidbar.

Wenn wir ein Problem haben auf das wir jedes Problem aus NP reduzieren können, dann gibt es kein Problem aus NP, das "schwieriger" ist. Ein solches Problem heißt NP-hart.

Definition 119: NP-Härte

Eine Sprache B heißt **NP-hart** oder NP-schwer (engl. NP-hard), falls für alle Sprachen L aus NP gilt

$$L \leq_P B$$

(B ist mindestens so schwierig wie jedes Problem in NP)

Wenn ein NP-hartes Problem selbst in NP liegt, muss es zu den schwierigsten Problemen in NP gehören.

Definition 120: NP-Vollständigkeit

Eine Sprache B heißt **NP-vollständig** (NP-complete), falls B in NP liegt und NP-hart ist.

3.5 Platzkomplexität

Definition 121: PSpace

Eine Sprache (Problem) ist in PSpace, genau dann wenn sie von einer deterministische Turing-Maschine mit polynomielltem Platzbedarf entschieden wird

Definition 122: NPSpace

Eine Sprache (Problem) ist in NPSpace, genau dann wenn sie von einer nichtdeterministische Turing-Maschine mit polynomielltem Platzbedarf entschieden wird

Ploynomialzeit Reduzierbarkeit
(Polynomialtime Reducability)

https://youtu.be/_BwhuUVj-1M
(<https://youtu.be/c1uwPLAz458>)

SAT ist NP-vollständig (SAT is NP-complete)

<https://youtu.be/TqGxiXV58XA>
(<https://youtu.be/rbUSAKJlhrM>)

NP-vollständige Probleme (NP complete Problems)

<https://youtu.be/gVfgzCRvXnI>
(<https://youtu.be/x99NCdfctug>)

Rucksack ist NP-vollständig
(Rucksack is NP Complete)

<https://youtu.be/0sb-iDQnk8g>
(https://youtu.be/hNQp2_0ae3I)

Beziehung zwischen Platz und Zeit (Relation Space / Time)

<https://youtu.be/sv-c2dJrln0>
(<https://youtu.be/nPj4wnWQyXM>)

Theorem 123: Satz von Savitch

Sei L eine Sprache, die von einer nichtdeterministischen Turing-Maschine mit mindestens linearem Platzbedarf $f(n) \geq n$ entschieden wird, dann gibt es eine deterministische Turing-Maschine mit Platzbedarf in $O(f(n)^2)$, die die Sprache entscheidet.

Theorem 124: PSpace = NPSpace

$\text{PSpace} = \text{NPSpace}$

Theorem 125: P, NP, PSpace

$P \subseteq NP \subseteq \text{PSpace}$

Definition 126: L

Eine Sprache (Problem) ist in L , genau dann wenn sie von einer deterministischen Turing-Maschine mit $\log n$ Platzbedarf entschieden wird

Definition 127: NL

Eine Sprache (Problem) ist in NL , genau dann wenn sie von einer nichtdeterministischen Turing-Maschine mit $\log n$ Platzbedarf entschieden wird

Definition 128: Log-Space Reduzierbarkeit

Ein Log-Space-Transducer ist eine Turing-Maschine mit einem read-only Eingabeband, einem write-only Ausgabeband und einem log-space Arbeitsband. Eine Sprache A heißt log-space reduzierbar auf B falls es eine totale mit log-space Platz – d.h. von einem Log-Space-Transducer – berechenbare Funktion f gibt mit $w \in A$ genau dann wenn $f(w) \in B$.

Platzverbrauch im deterministischen und nichtdeterministischen Fall (Deterministic Space and Non-deterministic Space)

https://youtu.be/8_-rINLTpoY
(<https://youtu.be/Uce7MZkGUPk>)

PSpace = NPSpace (PSpace = NPSpace)

<https://youtu.be/218yNxx6b8g>
(<https://youtu.be/nAhgTRh6YJ8>)

Sublinearer Platzverbrauch (Sub-linear Space)

<https://youtu.be/ghS0n33KNog>
(<https://youtu.be/BDu72Fe61-c>)

 $a^n b^n$ ist in L ($a^n b^n$ is in L)

<https://youtu.be/d5-tdqoFUYg>
(<https://youtu.be/WzFr7fpdQFg>)

PATH ist in NL (PATH is in NL)

<https://youtu.be/HxL8XQj7r0s>
(<https://youtu.be/eZow9qnPKm4>)

Zusammenfassung (Summary)

https://youtu.be/jQayJLI9_No
(<https://youtu.be/tfRd22bUPX0>)